

Process Virtualization

Sungyong Park, Ph.D

Data-Intensive Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

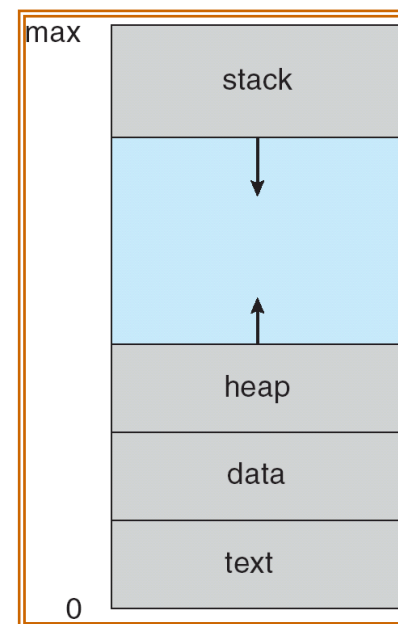
<https://discos.sogang.ac.kr>

Acknowledgement: Modified from lecture slides made by Prof. Youjip Won (KAIST)
based on the Operating Systems: Three Easy Pieces (OSTEP) textbook

4. The Abstraction: The Process

What is a Process?

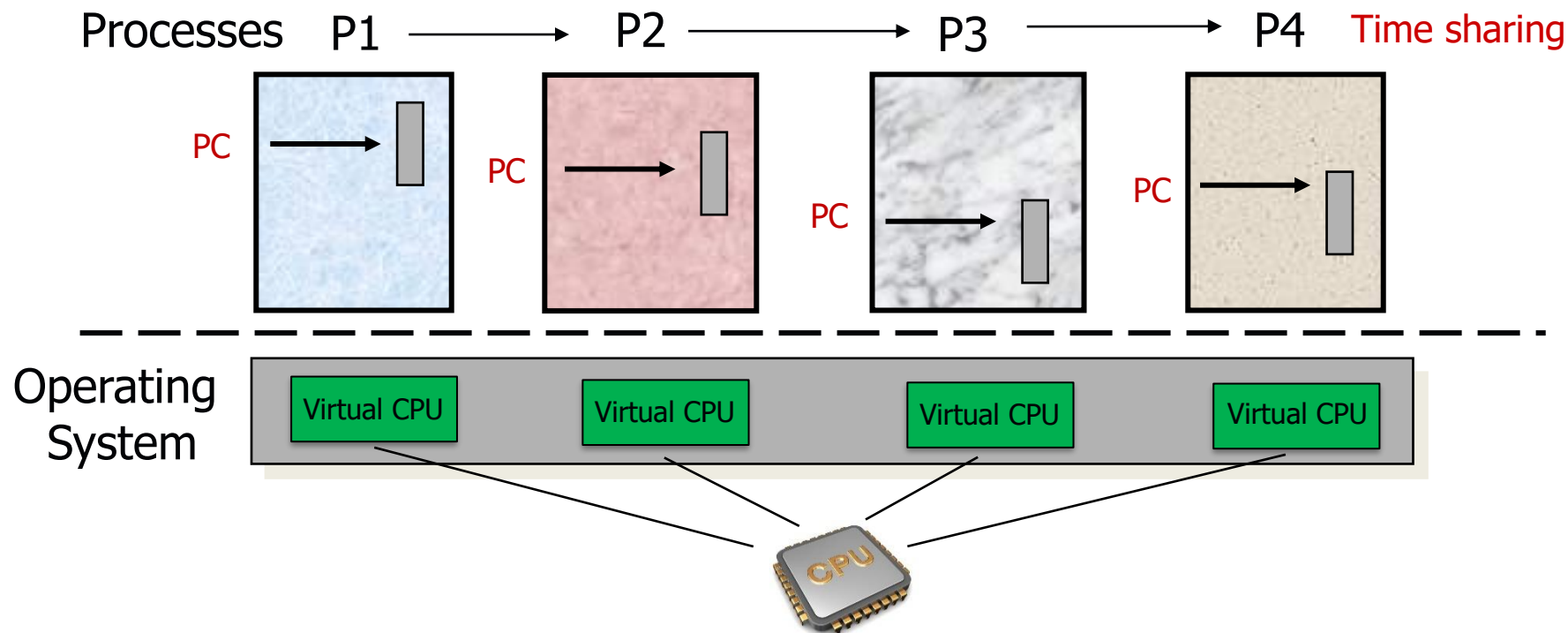
- ❑ Process is a **running program**.
- ❑ A **program** is a **passive** entity (e.g., a file containing a list of instructions stored on disks (executable file)).
- ❑ A **process** is an **active** entity with a program counter.
- ❑ A program becomes a process when an executable file is loaded into memory.
 - ◆ Multiple processes can be created by a program.
- ❑ We use *job*, *task* and *process* almost interchangeably.
- ❑ A process is comprised of **code** (or text) section, **data** section, **stack** and **heap**, and program counter.



How to Provide the Illusion of Many CPUs?

□ CPU virtualization

- ◆ The OS can promote the illusion that many virtual CPUs exist.
- ◆ **Time sharing**: Running one process, then stopping it and running another
 - The potential cost is **performance**.



- ▣ These APIs are available on any modern OS.
 - ◆ **Create**
 - Create a new process to run a program
 - ◆ **Destroy**
 - Halt a runaway process
 - ◆ **Wait**
 - Wait for a process to stop running
 - ◆ **Miscellaneous Control**
 - Some kind of method to suspend a process and then resume it
 - ◆ **Status**
 - Get some status info about a process

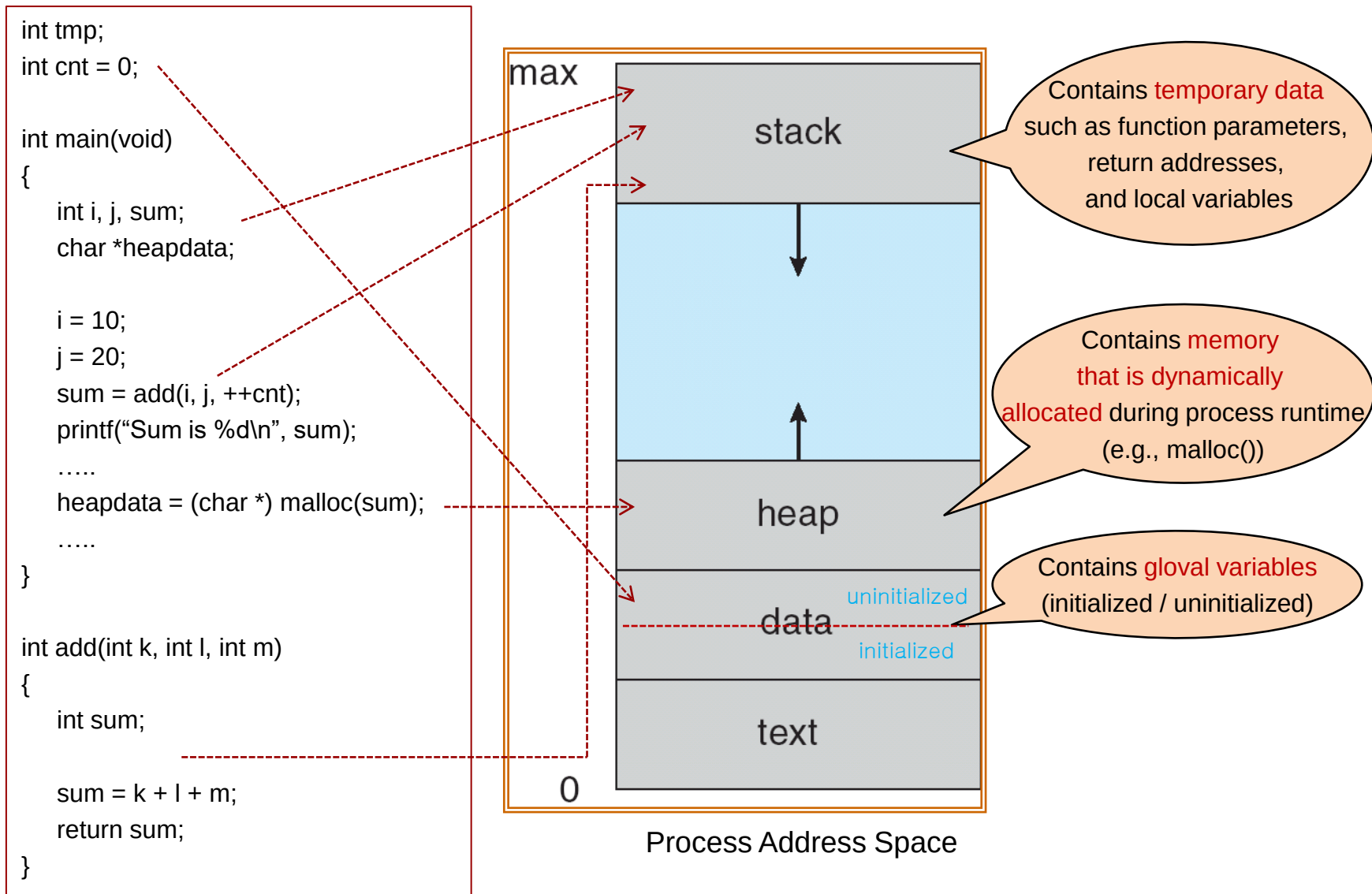
Process Creation

1. Load a program code into memory (into the address space of the process).
 - ◆ Programs initially reside on disk in *executable format*.
 - ◆ Modern OS perform the loading process **lazily**.
 - Loading pieces of code or data only as they are needed during program execution.
-> use virtual memory concept (will be discussed later).
2. The program's run-time stack is allocated.
 - ◆ Use the stack for *local variables*, *function parameters*, and *return address*.
 - ◆ Initialize the stack with arguments → `argc` and the `argv` array of `main()` function.

Process Creation (Cont.)

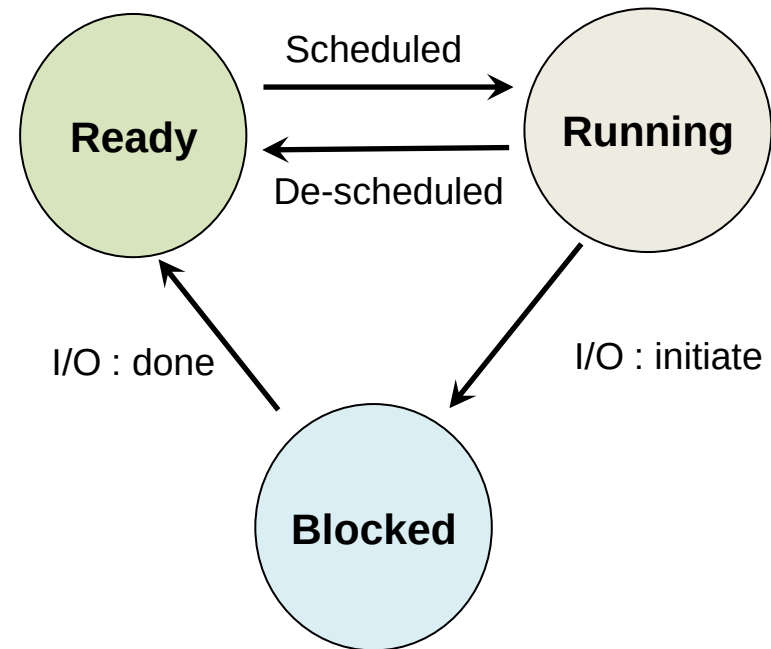
3. The program's heap is created.
 - ◆ Used for explicitly requested dynamically allocated data.
 - ◆ Program request such space by calling `malloc()` and free it by calling `free()`.
4. The OS do some other initialization tasks.
 - ◆ Input / Output (I/O) setup
 - Each process by default has three open file descriptors.
 - Standard input (*stdin*), standard output (*stdout*) and standard error (*stderr*).
5. Start the program running at the entry point, namely `main()`.
 - ◆ The OS *transfers control* of the CPU to the newly-created process.

Process in Memory



Process States and Transition

- A process can generally be one of three states (may have different states in other Operating Systems).
 - ◆ **Ready**
 - A process is ready to run but for some reason the OS has chosen not to run it at this given moment.
 - ◆ **Running**
 - A process is running on a processor.
 - ◆ **Blocked**
 - A process has performed some kinds of operations (e.g., I/O or event wait).
 - When a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.



Example : Tracing Process State

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process ₀ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process ₁ now done

Figure 4.3: Tracing Process State: CPU Only

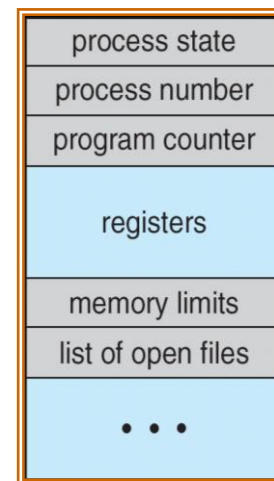
Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked, so Process ₁ runs
5	Blocked	Running	
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Data Structures

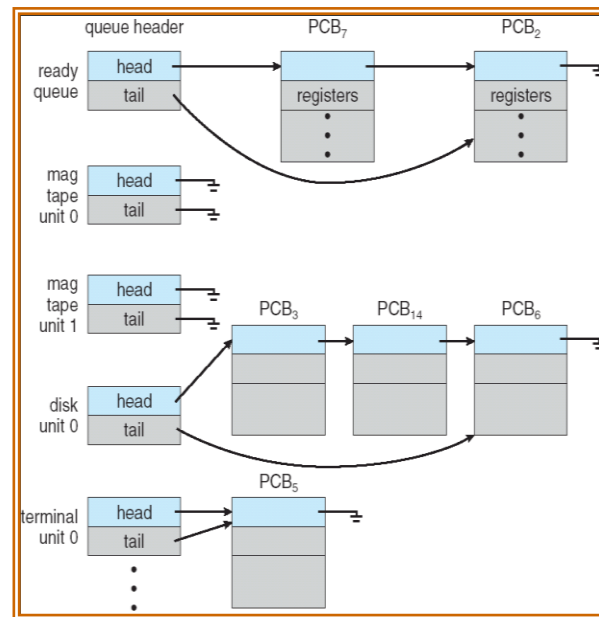
■ PCB (Process Control Block)

- ◆ A structure that contains information **about each process.**
- ◆ Includes *register context*: a set of registers that define the state of a process.
- ◆ Other information associated with each process.



■ Process List (Queue)

- ◆ Ready processes
- ◆ Blocked processes
- ◆ Current running processes



Example : The xv6 Kernel Proc Structure

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;           // Start of process memory
    uint sz;             // Size of process memory
    char *kstack;        // Bottom of kernel stack
                        // for this process
    enum proc_state state; // Process state
    int pid;             // Process ID
    struct proc *parent; // Parent process
    void *chan;          // If non-zero, sleeping on chan
    int killed;          // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;    // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf; // Trap frame for the
                        // current interrupt
};
```

Example : The xv6 Kernel Proc Structure (Cont.)

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;    // Index pointer register
    int esp;    // Stack pointer register
    int ebx;    // Called the base register
    int ecx;    // Called the counter register
    int edx;    // Called the data register
    int esi;    // Source index register
    int edi;    // Destination index register
    int ebp;    // Stack base pointer register
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```

Example : Linux task_struct

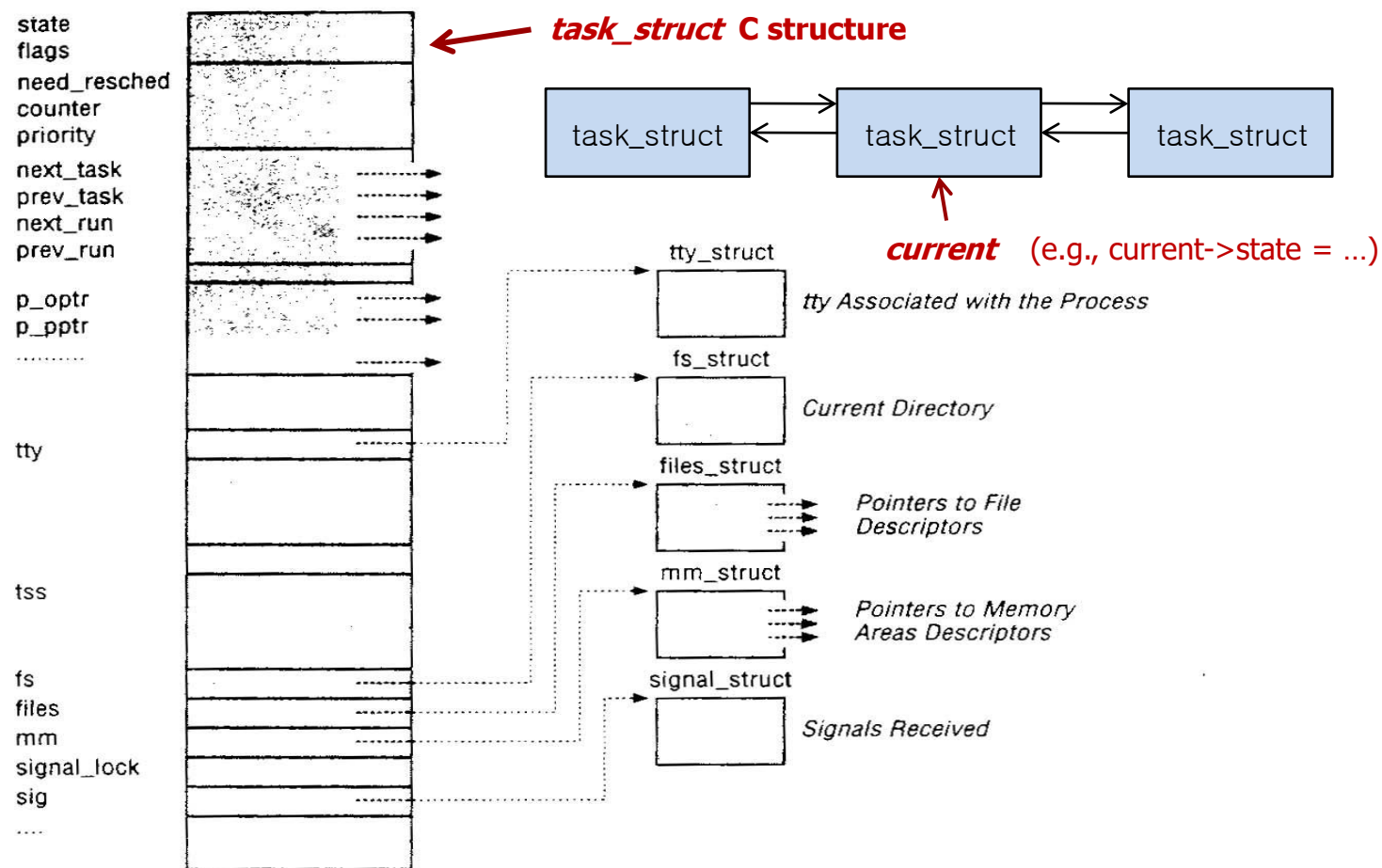
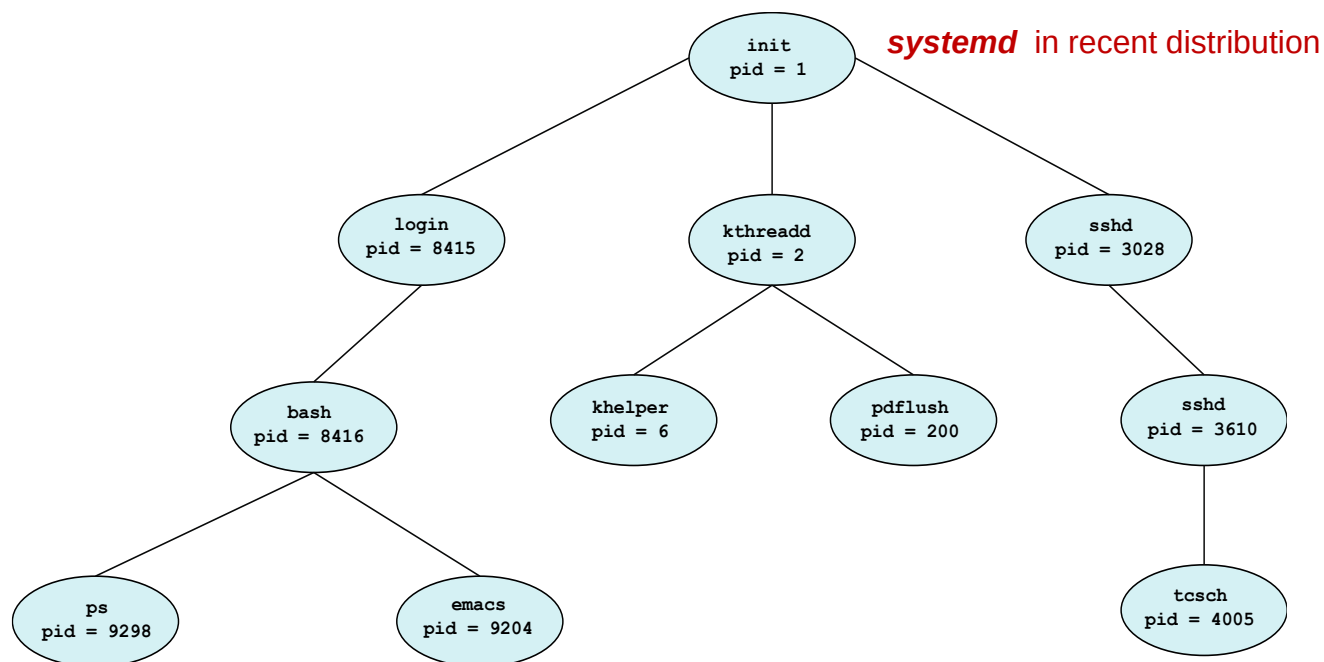


Figure 3-1. The Linux process descriptor

5. Process API

Process ID

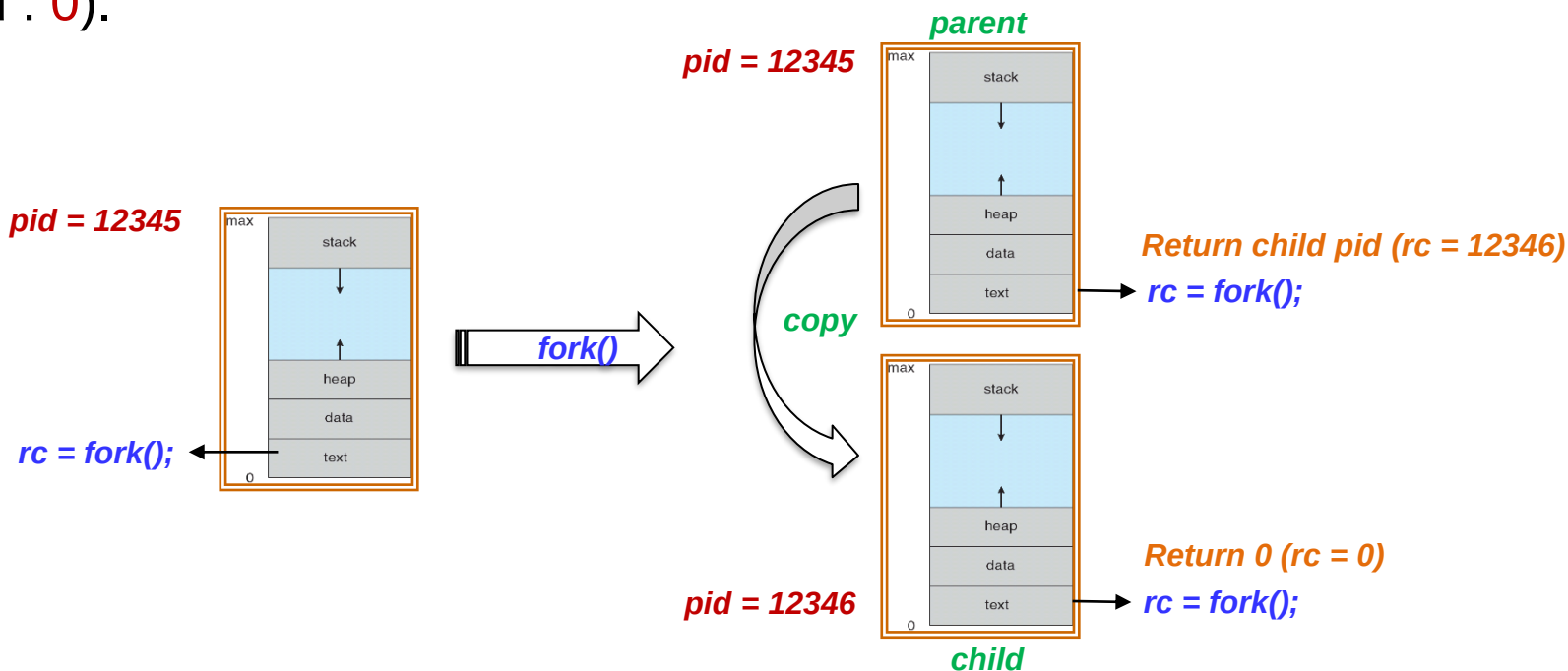
- ❑ Process is identified and managed via a process identifier (**pid**).
- ❑ Parent process creates children processes, which, in turn create other processes, forming a tree of processes.
- ❑ Process tree in a Linux system :



Creating a Process

`fork()`

- Parent process uses a `fork()` system call to create a child process.
- The child process consists of a copy of the address space of the parent.
- Both processes continue execution at the instruction **after** `fork()` with one difference: **Returned values are different** (i.e., parent : **child pid**, child : **0**).



Usage of `fork()`

p1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {          // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {              // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int)
getpid());
    }
    return 0;
}
```

Let's Run

```
prompt> ./p1  
hello world (pid:29146)  
hello, I am parent of 29147 (pid:29146)  
hello, I am child (pid:29147)  
prompt>
```

or

```
prompt> ./p1  
hello world (pid:29146)  
hello, I am child (pid:29147)  
hello, I am parent of 29147 (pid:29146)  
prompt>
```

Reaping a Child Process

`wait()` or `waitpid()`

- The `wait(&status)` system call allows the child process to **report an exit status** (`exit(status)`) back to the parent process when it terminates.
- This exit status is stored in the process table and the process table entry is deallocated after the parent process invokes `wait()` call.
- If the parent process fails to invoke `wait()` call, the process table entry for the child process **remains** in the table, which creates a **zombie** process.
- **Reaping** is performed by parent using `wait()` or `waitpid()` system calls where parent is given exit status and OS removes zombie child process from the process table.

Usage of `wait()`

p2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {                    // parent goes down this path (main)
        int wc = wait(NULL);    // Reaping child process
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

Let's Run

```
prompt> ./p2  
hello world (pid:29266)  
hello, I am child (pid:29267)  
hello, I am parent of 29267 (wc:29267) (pid:29266)  
prompt>
```

Zombie vs. Orphan Processes

- A **zombie** process or **defunct** process is a process that has completed execution but still has an entry in the process table.
 - ◆ A process that has terminated, but its parent has not yet collected the status via *wait* -> usually because of bugs or coding errors.
 - ◆ All processes can normally exist as zombies only briefly.
 - ◆ When the parent calls *wait*, the child terminates normally.
 - ◆ Parent is executing, while the child is dead.
- When a parent didn't invoke *wait* and instead terminated, its children processes are **orphan** processes (i.e., still executing).
 - ◆ The *init* process is assigned to the new parent of those processes.
 - ◆ The *init* process periodically invokes *wait* to collect the exit status of any orphan processes.
 - ◆ Parent (original) is dead, while the child is executing.

Running a New Program

`exec()`

- ❑ The caller wants to run a program that is different from the caller itself.
 - ◆ Launch an editor
 - ◆ `% ls -l`
- ❑ OS needs to load a new binary image, initialize a new stack, initialize a new heap for the new program.
- ❑ two parameters
 - ◆ The name of the binary file
 - ◆ The array of arguments
- ❑ Replace the existing contents of the memory with the new memory contents from the new binary file.
- ❑ `exec()` does not return. It starts to execute the new program.

```
char *argv[3];  
  
argv[0] = "echo";  
argv[1] = "hello";  
argv[2] = 0;  
exec("/bin/echo", argv);  
printf("exec error\n");
```



Usage of `exec()`

p3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[]){
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {                                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {                        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc");                // program: "wc" (word count)
        myargs[1] = strdup("p3.c");              // argument: file to count
        myargs[2] = NULL;                        // marks end of array
        execvp(myargs[0], myargs);                // runs word count
        printf("this shouldn't print out");
    } else {                                    // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
            rc, wc, (int) getpid());
    }
    return 0;
}
```

Let's Run

Result

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
29 107 1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

Why Separating `fork()` and `exec()`?

- ❑ Why don't we just use something like “`forkandexec("ls", "ls -l")`”?
- ❑ Via separating `fork()` and `exec()`, we can manipulate various settings just before executing a new program and make the IO redirection and pipe possible.



◆ IO redirection

```
% cat w3.c > newfile.txt
```

◆ pipe

```
% echo hello world | wc
```

‘pipe’ is the heart of the shell programming.

What is IO Redirection?

- IO redirection means capturing output from a file, command, program or script and sending it as input to another file, command, program or script.

- Example: Save the result of `'wc w3.c'` to `newfile.txt`.

```
% wc w3.c > newfile.txt
```

How?

- Shell is a program that forks (`fork()`) the command and executes (`exec()`) it with argument.
 - Shell calls `fork()` and `exec("wc", "wc w3.c")`.
 - Before calling `exec("wc", "wc w3.c")`, the shell **closes** `STDOUT` and **opens** `newfile.txt` (`open("newfile.txt")`).

Example: IO Redirection

p4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>

int
main(int argc, char *argv[]){
    int rc = fork();
    if (rc < 0) {                                // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {                        // child: redirect standard output to a file
        close(STDOUT_FILENO);
        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
        // now exec "wc"...

        char *myargs[3];
        myargs[0] = strdup("wc");                // program: "wc" (word count)
        myargs[1] = strdup("p4.c");              // argument: file to count
        myargs[2] = NULL;                        // marks end of array
        execvp(myargs[0], myargs);               // runs word count
    } else {                                    // parent goes down this path (main)
        int wc = wait(NULL);
    }
    return 0;
}
```

Let's Run

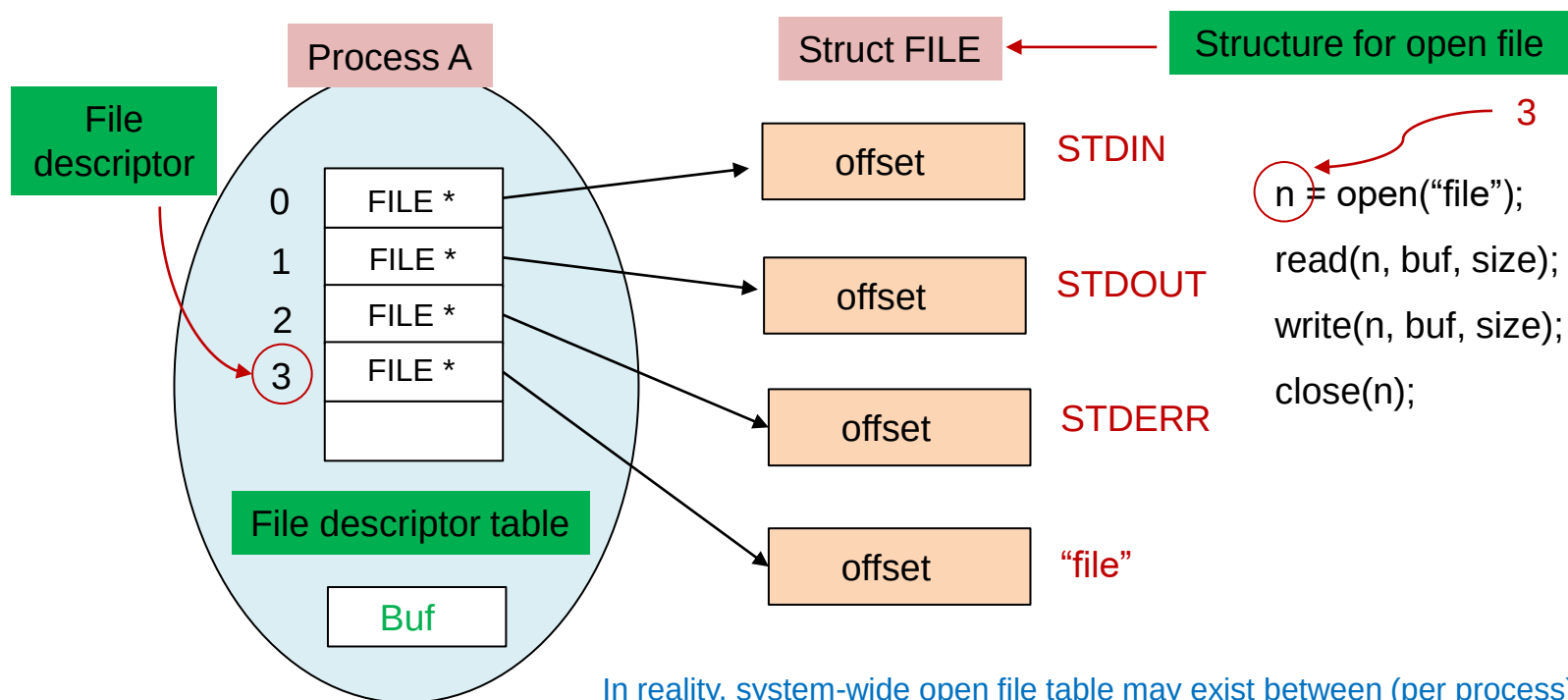
Result

```
prompt> ./p4  
prompt> cat p4.output  
32 109 846 p4.c  
prompt>
```

File Descriptor and File Descriptor Table

File descriptor

- File descriptor is an integer that represents a file, a directory or a device.
- A process uses a file descriptor to open a file, a directory or a device.
- Each process has its own file descriptor table.
- Standard Streams (opened by default) : File descriptor 0 (Standard Input - STDIN), 1 (Standard Output - STDOUT), 2 (Standard Error - STDERR)



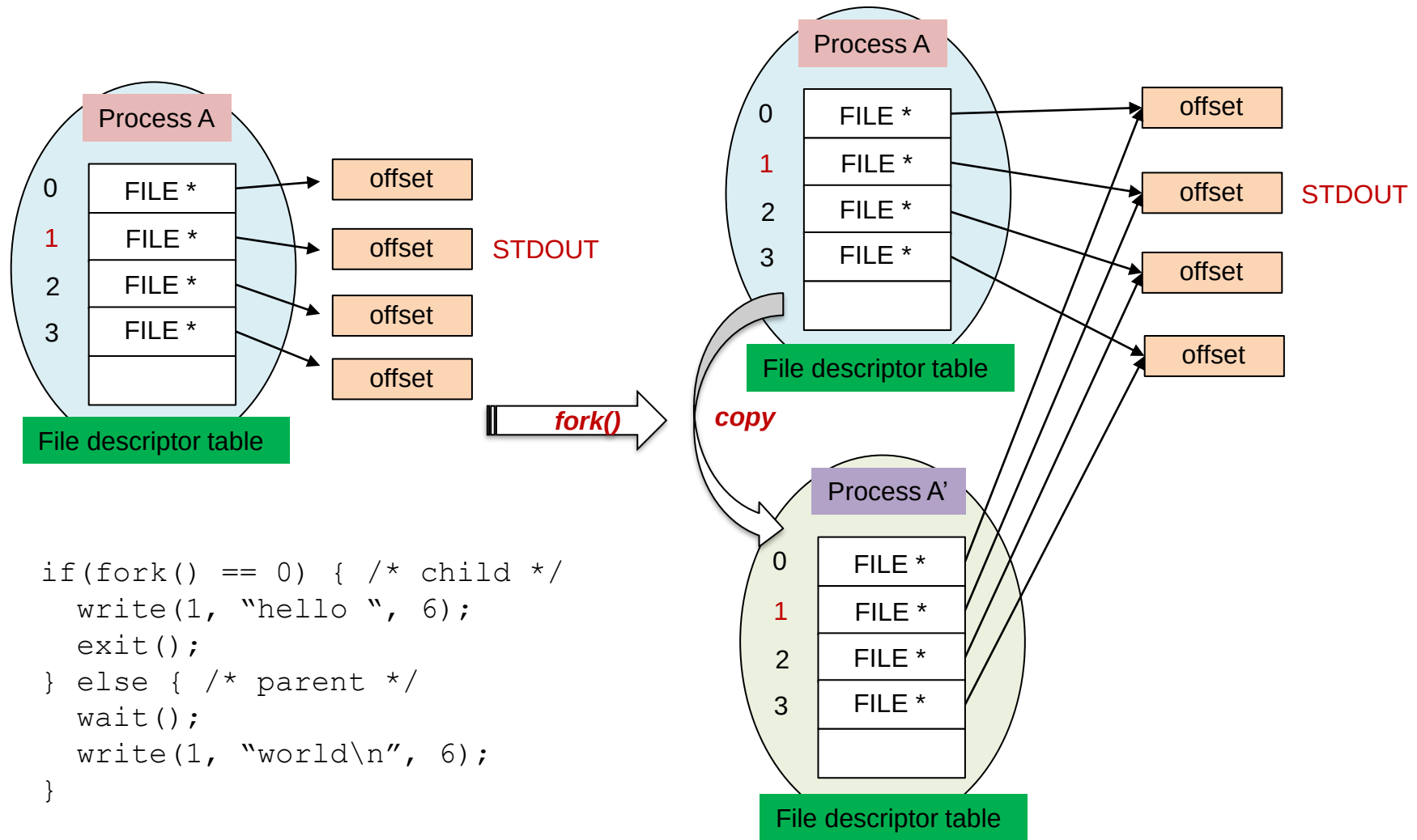
In reality, system-wide open file table may exist between (per process) file descriptor table and FILE structure !

File Descriptor and System Calls

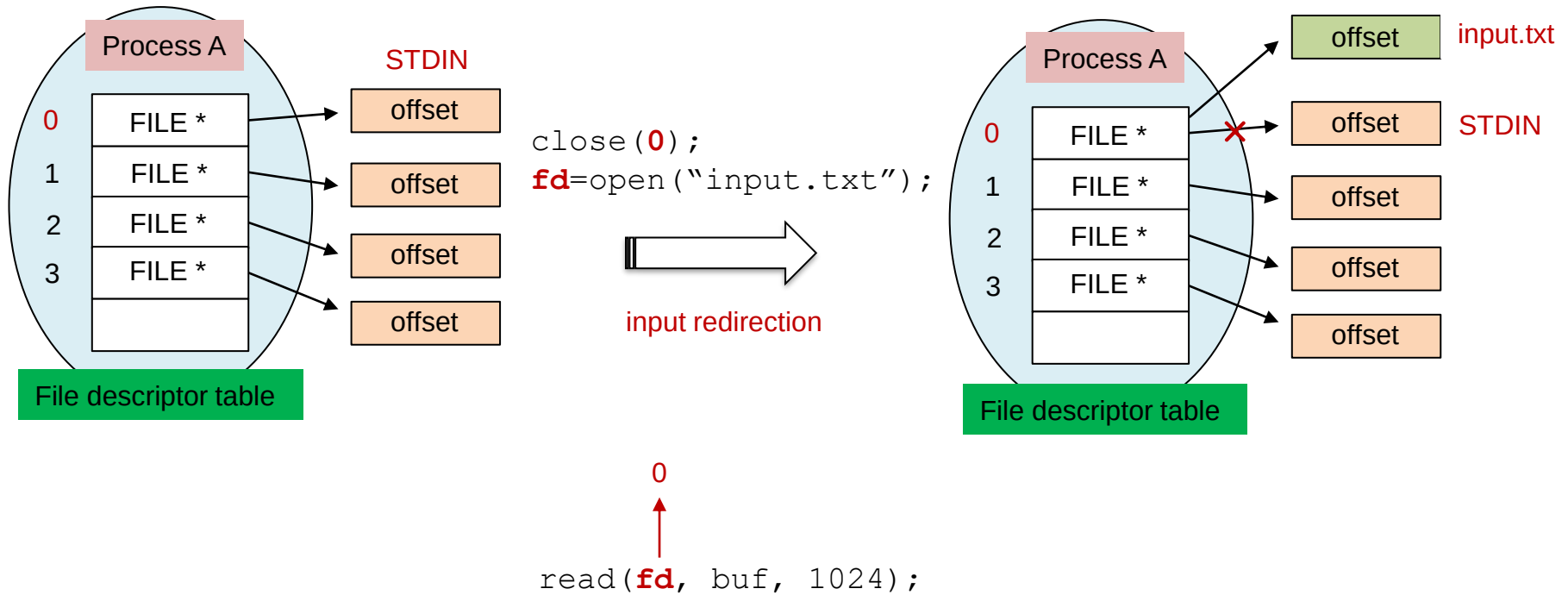
- `open ()`
 - ◆ Allocate a new file object, allocate new file descriptor and set the newly allocated file descriptor to point to the new file object.
 - ◆ When allocating the new file descriptor, it uses the **smallest 'free' file descriptor** from the file descriptor table.
- `close ()`
 - ◆ Deallocate the file descriptor 'fd'.
 - ◆ Deallocate the file object if there is no file descriptor associated with it.
- `fork ()`
 - ◆ **Copies the file descriptor table** from the parent to child process.
- `exec ()`
 - ◆ **Retains the file descriptor table.**



fork() and File Descriptors

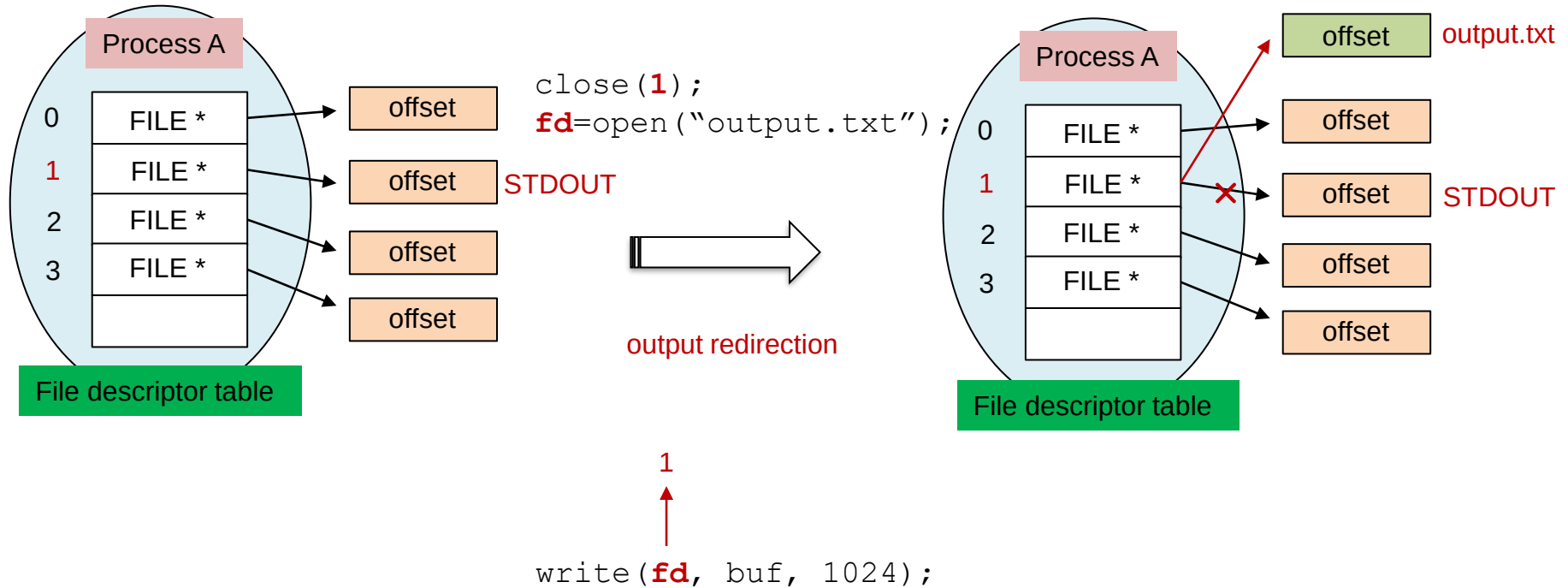


Input Redirection



Read 1024 bytes from "input.txt" !

Output Redirection



Write the content of buf to "output.txt" !

Example : cat and IO Redirection

```
% cat input.txt
```

```
char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if(fork() == 0) {
    close(0);
    open("input.txt", O_RDONLY);
    exec("cat", argv);
}
```

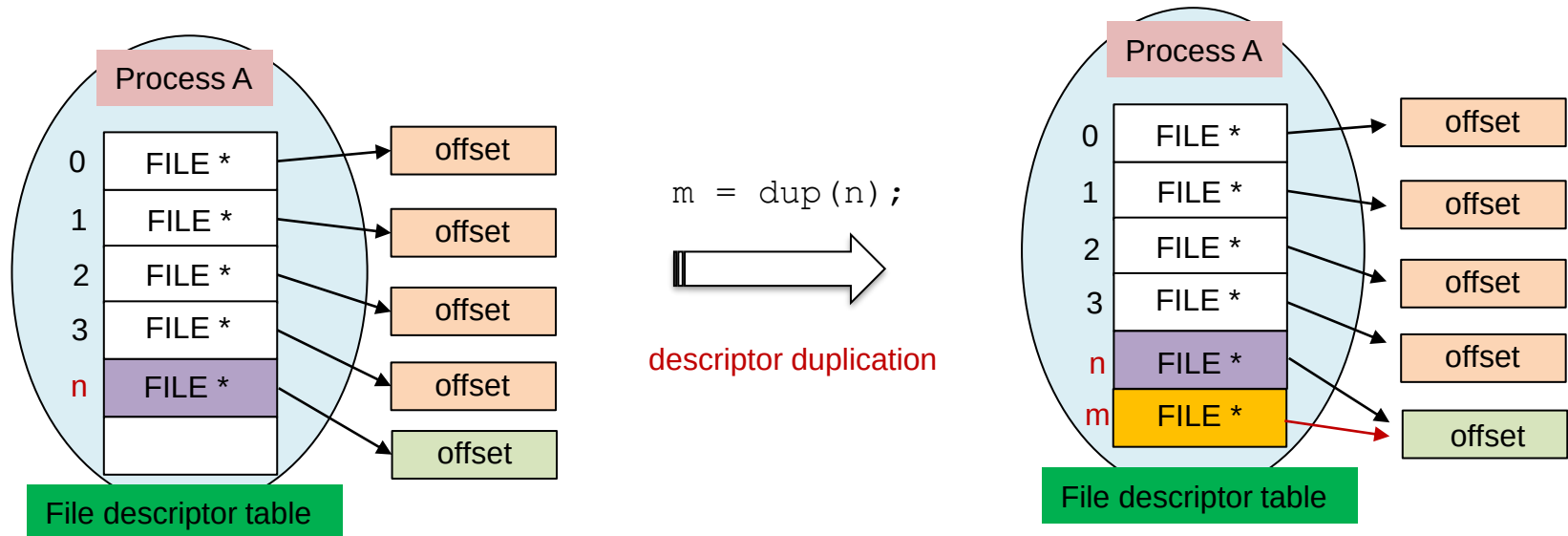
Redirect standard input to "input.txt".

```
char buf[512];
int n;

for(;;) {
    Standard Input -> "input.txt"
    n = read(0, buf, sizeof(buf));
    if(n == 0)
        break;
    if(n < 0) {
        Standard Error
        fprintf(2, "read error\n");
        exit();
    }
    Standard Output
    if(write(1, buf, n) != n) {
        fprintf(2, "write error\n");
        exit();
    }
}
```

Duplication of File Descriptor using `dup()`

`dup(n)`: Find an empty slot from the beginning of the file descriptor table.

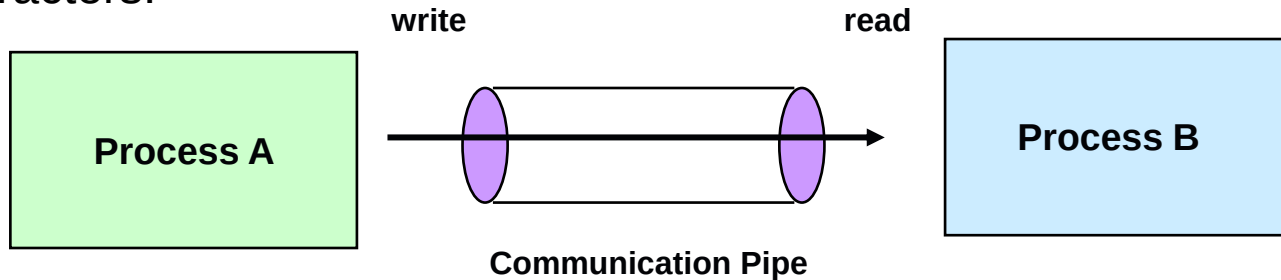


```
fd = dup(1);  
write(1, "hello ", 6);  
write(fd, "world\n", 6);
```

Write "hello world" to the standard output !

What is Pipe?

- ▣ *Unidirectional* byte streams which connect one process into the other process.
 - ◆ The data written at one end of the channel is read at the other end.
 - ◆ From a logical point of view, a pipe can be compared to a FIFO queue of characters.



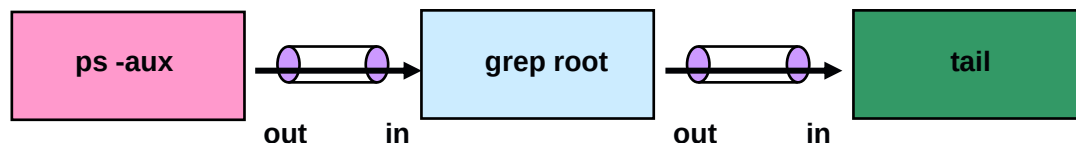
- ◆ No structured communication : it is not possible to know the size, sender/receiver of data contained in the pipe.
- ◆ Pipe is a special type of file, a kernel buffer that is exposed to a process via file descriptors.
- ◆ Access to pipes is achieved by reading/writing from/to file descriptors.

Pipe Used by Commands

- One current usage of the pipe mechanism is performed by means of the command line interpreter when commands are linked:

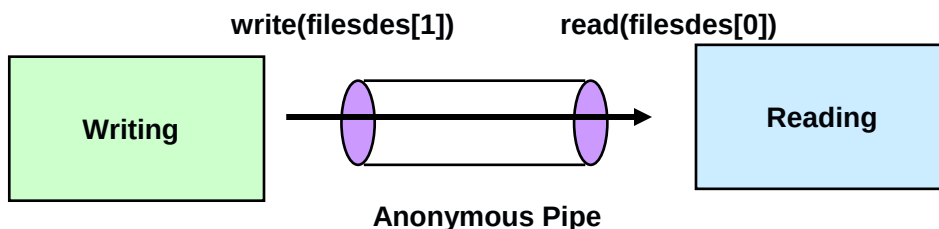
(e.g., `> ps -aux | grep root | tail`)

```
[ /home/discos ] ps -aux | grep root | tail
root  9232 0.0 0.1 1392 664 ?      S   Aug26   0:00 in.identd -e -o
root  9233 0.0 0.1 1392 664 ?      S   Aug26   0:00 in.identd -e -o
.....
root  16763 0.2 0.1 1704 876 ?      S   10:32   0:00 in.telnetd
root  16764 0.1 0.2 2320 1232 pts/0  S   10:32   0:00 login - discos
discos 16800 0.0 0.1 2236 528 pts/0  S   10:33   0:00 grep root
```

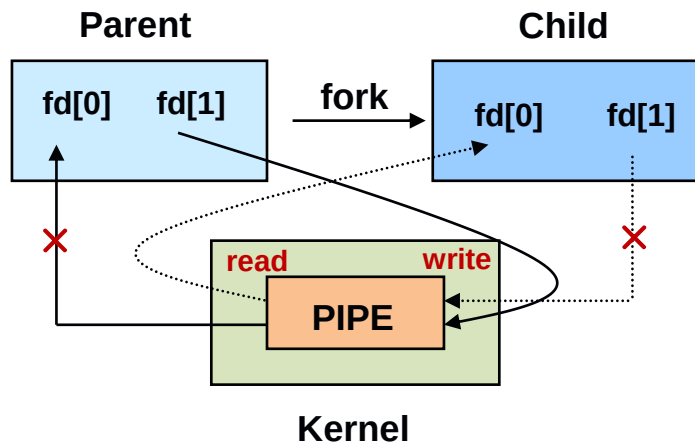


Anonymous Pipe

- Created by a process and the transmission for associated descriptors is achieved only by inheritance by its descendants. (i.e., by creating a child process using `fork()` system call)
- Restrictive in that it only allows communication between processes with a common ancestor which is a creator of a pipe.
- Creation of an anonymous pipe :
 - `int pipe(int filesdes[2]);`
 - > `filesdes[0]` : **read** descriptor, `filesdes[1]` : **write** descriptor.



Usage of `pipe()`



```
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int n, fd[2], pid; char line[100];
    if (pipe(fd) < 0) exit(-1);
    if ((pid = fork()) < 0) exit(-1);
    else if (pid > 0) { /*parent */
        close(fd[0]);
        write(fd[1], "Hello World\n", 12);
        close(fd[1]);
        wait(NULL);
    }
    else { /* child */
        close(fd[1]);
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }
}
```

Named Pipe (FIFO)

- ❑ Communication is bi-directional, and no parent-child relationship.
- ❑ Entries related pipes are managed in the file system.
- ❑ Named pipes continue to exist after communicating processes have finished unless the pipe is unlinked.
- ❑ Has a name and handled exactly like files with respect to file operations (e.g., open, close, read, write).
- ❑ Created by *mkfifo* or *mknod* commands.
- ❑ Can be created and removed by C functions : *mkfifo()* / *unlink()*
 - ◆ *int mkfifo(const char *path, mode_t mode);*
 - ◆ *int unlink(const char *path);*
- ❑ Reading from / writing to a named pipe can be achieved by using standard *read()* and *write()* system calls.

Usage of `mkfifo()`

writer.c

```
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    int fd; char * myfifo = "/tmp/myfifo";

    /* create the FIFO (named pipe) */
    mkfifo(myfifo, 0666);

    /* write "Hi" to the FIFO */
    fd = open(myfifo, O_WRONLY);
    write(fd, "Hi", sizeof("Hi"));
    close(fd);

    /* remove the FIFO */
    unlink(myfifo);
    return 0;
}
```

reader.c

```
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>
#define MAX_BUF 1024
int main()
{
    int fd; char * myfifo = "/tmp/myfifo";
    char buf[MAX_BUF];

    /* open, read, and display the message
       from the FIFO */
    fd = open(myfifo, O_RDONLY);
    read(fd, buf, MAX_BUF);
    printf("Received: %s\n", buf);
    close(fd);

    return 0;
}
```

Open Write-Only with and without O_NONBLOCK

- When opening a FIFO with O_RDONLY or O_WRONLY set:
 - ◆ If O_NONBLOCK is set, an `open()` with O_RDONLY shall return without delay. An `open()` with O_WRONLY shall return an error if no process currently has the file open for reading.
 - ◆ If O_NONBLOCK is clear, an `open()` with O_RDONLY or O_WRONLY shall block the calling thread until a thread opens the file for writing or reading.

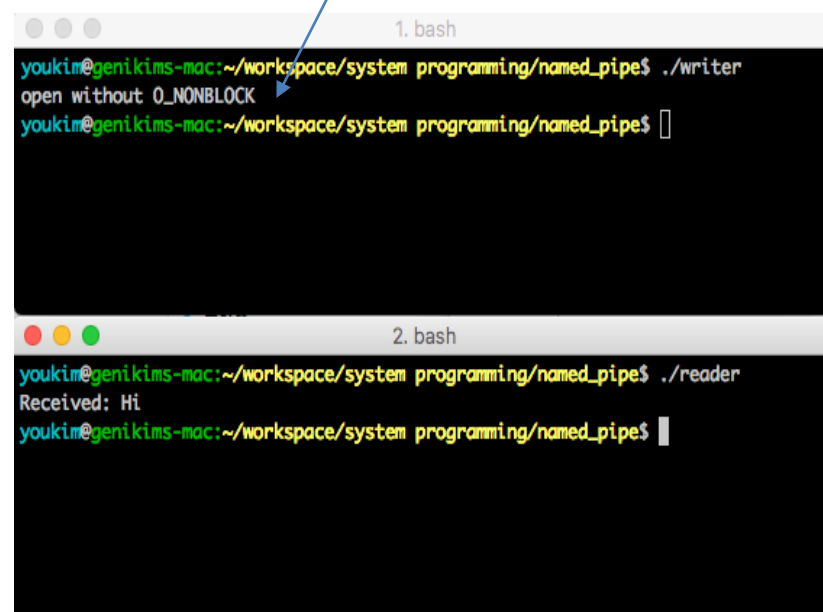
```
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    int fd; char * myfifo = "/tmp/myfifo";

    /* create the FIFO (named pipe) */
    mkfifo(myfifo, 0666);

    /* write "Hi" to the FIFO */
    printf("open without O_NONBLOCK\n");
    fd = open(myfifo, O_WRONLY);
    write(fd, "Hi", sizeof("Hi"));
    close(fd);

    /* remove the FIFO */
    unlink(myfifo);
    return 0;
}
```

Waiting for another process to open the FIFO

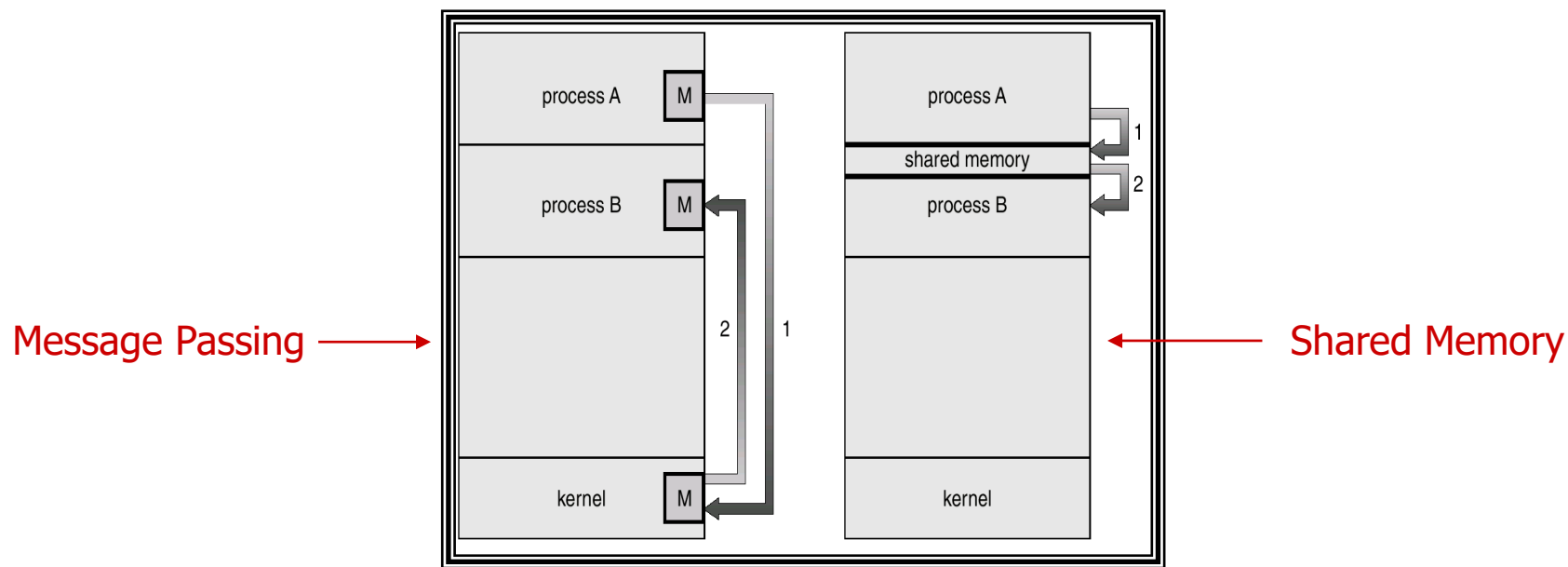


```
1. bash
youkim@genikims-mac:~/workspace/system programming/named_pipe$ ./writer
open without O_NONBLOCK
youkim@genikims-mac:~/workspace/system programming/named_pipe$

2. bash
youkim@genikims-mac:~/workspace/system programming/named_pipe$ ./reader
Received: Hi
youkim@genikims-mac:~/workspace/system programming/named_pipe$
```

Inter-Process Communication (IPC)

- Different processes run on different address space.
- Need a mechanism for various processes to communicate among them → IPC.
- Can be generally achieved via message passing or shared memory.
- Types of IPCs in UNIX
 - ◆ Traditional UNIX IPCs : **signal, pipe, socket**
 - ◆ System V IPCs : **message queue, semaphore, shared memory**



Example: POSIX Shared Memory (Producer)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```

Example: POSIX Shared Memory (Consumer)

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

6. Mechanism: Limited Direct Execution

Recap: Process Creation

1. The OS **loads** a program code into memory, into the address space of the process → becomes a process.
 - ◆ Programs initially reside on disk in *executable format* (e.g, `a.out`)
 - ◆ The OS perform the loading process **lazily**.
 - Loading pieces of code or data only as they are needed during program execution → called *demand paging* or *demand segmentation* (will be covered later).
2. The program's run-time **stack** is allocated.
 - ◆ Use the stack (or registers) for *local variables*, *function parameters*, and *return address*.
 - ◆ Initialize the stack with arguments → `argc` and the `argv` array of `main()` function.

Recap: Process Creation (Cont.)

3. The program's **heap** is created.
 - ◆ Used for explicitly requested dynamically allocated data.
 - ◆ Program request such space by calling `malloc()` and free it by calling `free()`.
4. The OS does some other **initialization** tasks.
 - ◆ Input/Output (I/O) setup.
 - Each process by default has three open file descriptors.
 - Standard input (STDIN), output (STDOUT) and error (STDERR).
5. The OS **starts the program** running at the entry point, namely `main()`.
 - ◆ The OS *transfers control* of the CPU to the newly-created process.

Direct Execution

- Just run the program directly on the CPU for **performance** reason without any limits (or control).

OS	Program
1. Create entry for process list. 2. Allocate memory for program. 3. Load program into memory. 4. Set up stack with <code>argc / argv</code>. 5. Clear registers. 6. Execute call <code>main()</code>. 9. Free memory of process. 10. Remove from process list.	7. Run <code>main()</code>. 8. Execute return from <code>main()</code>.

**Without *limits* on running programs,
the OS wouldn't be in control of anything and
thus would be "just a library"**

Any Problems in Direct Execution?

▣ Problem #1 : Restricted Operations

- ◆ What if a process wishes to perform some kinds of restricted operations such as
 - Issuing an I/O request to a disk.
 - Gaining access to more system resources such as CPU or memory.
- ◆ Can we allow any process to do whatever it wants to do?
 - User can do the wrong thing.

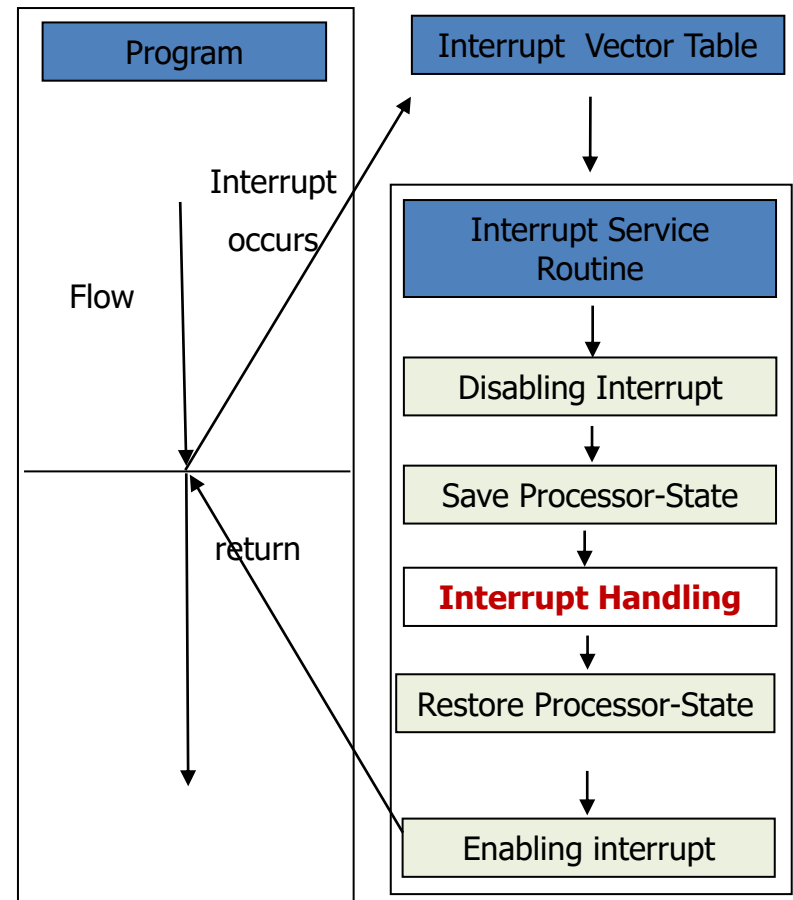
▣ Problem #2 : Switching Between Processes

- ◆ Getting the control back from CPU is not easy.
- ◆ What if?

```
i = -1 ;  
while ( i < 0 )  
    do something;
```

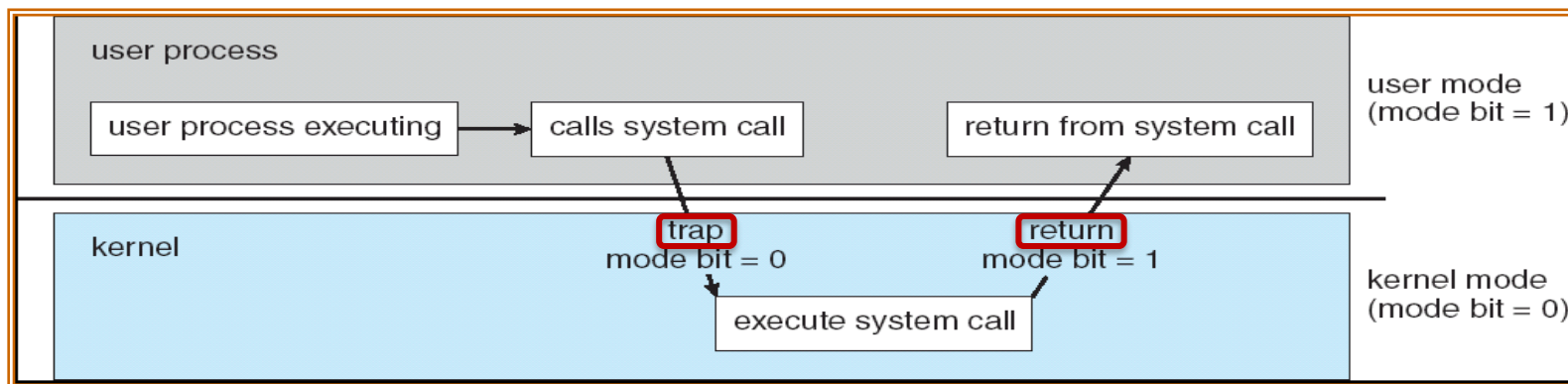
Recap: Interrupt Handling (SW Perspective)

- Interrupt transfers control to the **interrupt service routine** (or **interrupt handler**), through the **interrupt vector**, which contains the addresses of all the service routines (generally stored in low memory).
- Interrupt handler is generally a part of the operating system (generally set at boot time).
- Interrupt architecture must save the address of the interrupted instruction.
- A **trap** (or an **exception**) is a software-generated interrupt caused either by an error or a user request.



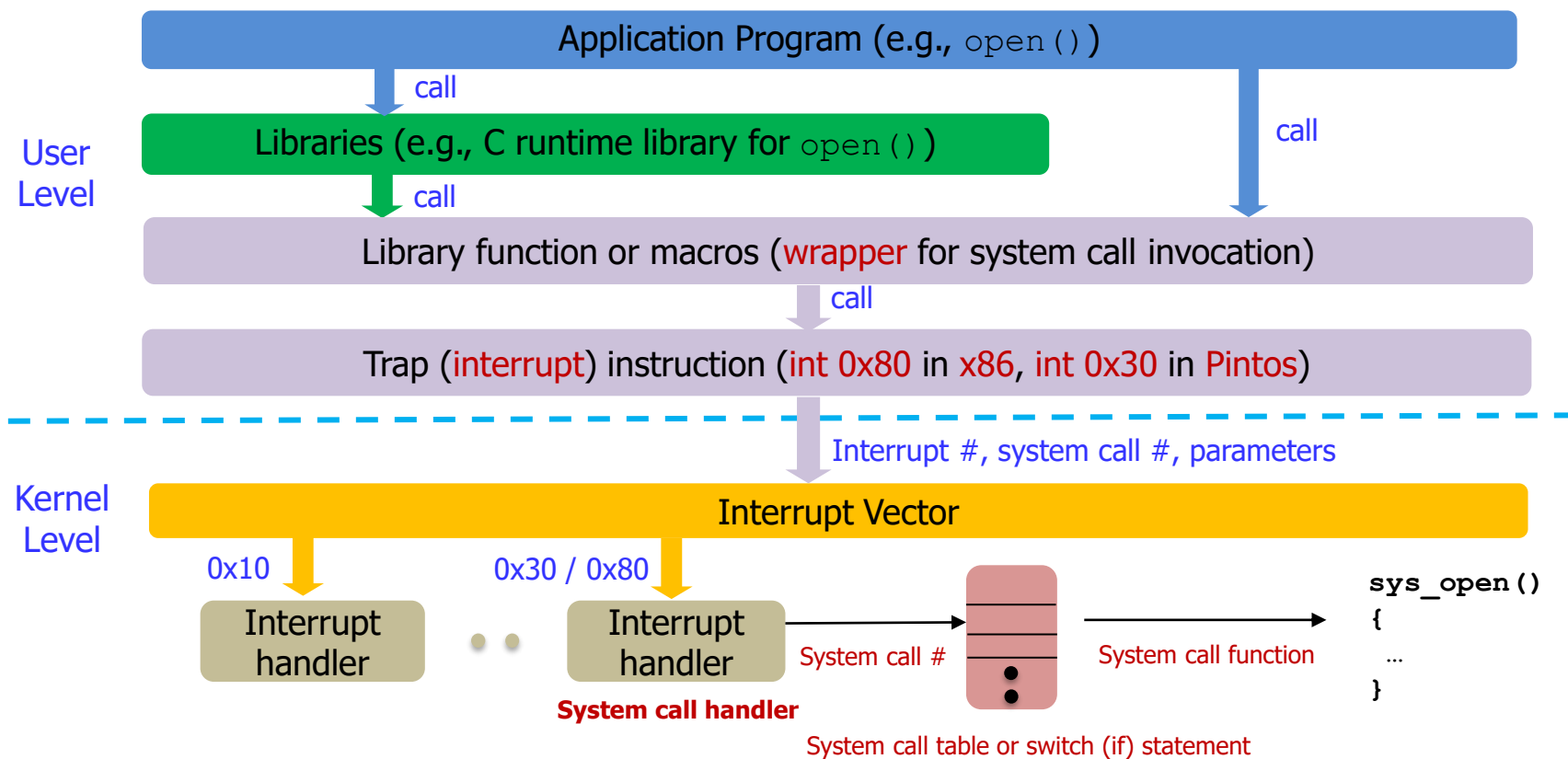
Solution for Problem 1: Dual Mode Operation and System Call

- **Dual-mode** operation allows OS to protect itself and other system components.
 - ◆ **user mode** and **kernel mode** (supervisor mode / system mode / privileged mode).
- Some instructions designated as **privileged**, are only executable in kernel mode.
 - ◆ A process in user mode can't issue privileged operations (such as I/O requests). If it does, the processor raises an exception and the OS would kill the process.
- Users invoke **system calls** to ask OS to do privileged operations. System calls change mode to kernel, return from call resets it to user.
 - ◆ The system call functions internally invoke trap (interrupt) instruction and the OS codes invoke return from trap instruction to change mode.
- The mode is managed via hardware **mode bit**.



Overall Flow of System Call

- ❑ Users invoke the kernel's key pieces of functionality via **library functions**.
- ❑ The caller needs to know nothing about how the system call is implemented.
 - ◆ Just needs to obey the API (Application Programming Interface) (e.g., POSIX API or Win64 API, etc.) and understand what OS will do as a result of the call.
 - ◆ Most details of OS interface are hidden from programmer by API.



Example: Windows / Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

System Call Parameter Passing

- Three general methods are used to pass parameters to the OS.
 - ◆ Simplest: pass the parameters in **registers**.
 - In some cases, there may be more parameters than the number of registers.
 - ◆ Parameters are stored in a **block or table** in memory and the address of the block is passed as a parameter in a register.
 - ◆ Parameters are placed or **pushed onto the stack** by the program and **popped off** the stack by the operating system.
 - ◆ Block and stack methods do not limit the number or length of parameters being passed.
- Each OS has a different mechanism to pass parameters to the OS (called Application Binary Interface (ABI)).

Example: `write()` Flow in Pintos (User Side)

User program

```
main()
{
    write(".....")
}
```

pintos/src/lib/user/syscall.c

```
int write(...)
{
    return syscall13(SYS_WRITE, fd, buffer, size);
}

...
syscall13(...)
{
    ...

    push arg2
    push arg1
    push arg0
    push number
    int 0x30
    addl $16, esp
    Restore stack pointer
}
```

Macro

```
/* Invokes syscall NUMBER, passing arguments ARG0, ARG1, and
   ARG2, and returns the return value as an `int'. */
#define syscall13(NUMBER, ARG0, ARG1, ARG2)
({
    int retval;
    asm volatile
    ("pushl %[arg2]; pushl %[arg1]; pushl %[arg0]; "
     "pushl %[number]; int $0x30; addl $16, %%esp"
     : "=a" (retval)
     : [number] "i" (NUMBER),
       [arg0] "g" (ARG0),
       [arg1] "g" (ARG1),
       [arg2] "g" (ARG2)
     : "memory");
    retval;
})
```

System call number Parameters

User stack

Stack pointer (esp)

number
arg0
arg1
arg2

Example: Other System Calls (src/lib/user/syscall.c)

```
int
open (const char *file)
{
    return syscall1 (SYS_OPEN, file);
}

int
filesize (int fd)
{
    return syscall1 (SYS_FILESIZE, fd);
}

int
read (int fd, void *buffer, unsigned size)
{
    return syscall3 (SYS_READ, fd, buffer, size);
}

int
write (int fd, const void *buffer, unsigned size)
{
    return syscall3 (SYS_WRITE, fd, buffer, size);
}

void
seek (int fd, unsigned position)
{
    syscall2 (SYS_SEEK, fd, position);
}
```

Example: Type of System Calls in Pintos (syscall-nr.h)

```
/* System call numbers. */
enum
{
    /* Projects 2 and later. */
    SYS_HALT,           /* Halt the operating system. */
    SYS_EXIT,           /* Terminate this process. */
    SYS_EXEC,           /* Start another process. */
    SYS_WAIT,           /* Wait for a child process to die. */
    SYS_CREATE,         /* Create a file. */
    SYS_REMOVE,         /* Delete a file. */
    SYS_OPEN,           /* Open a file. */
    SYS_FILESIZE,       /* Obtain a file's size. */
    SYS_READ,           /* Read from a file. */
    SYS_WRITE,          /* Write to a file. */
    SYS_SEEK,           /* Change position in a file. */
    SYS_TELL,           /* Report current position in a file. */
    SYS_CLOSE,          /* Close a file. */

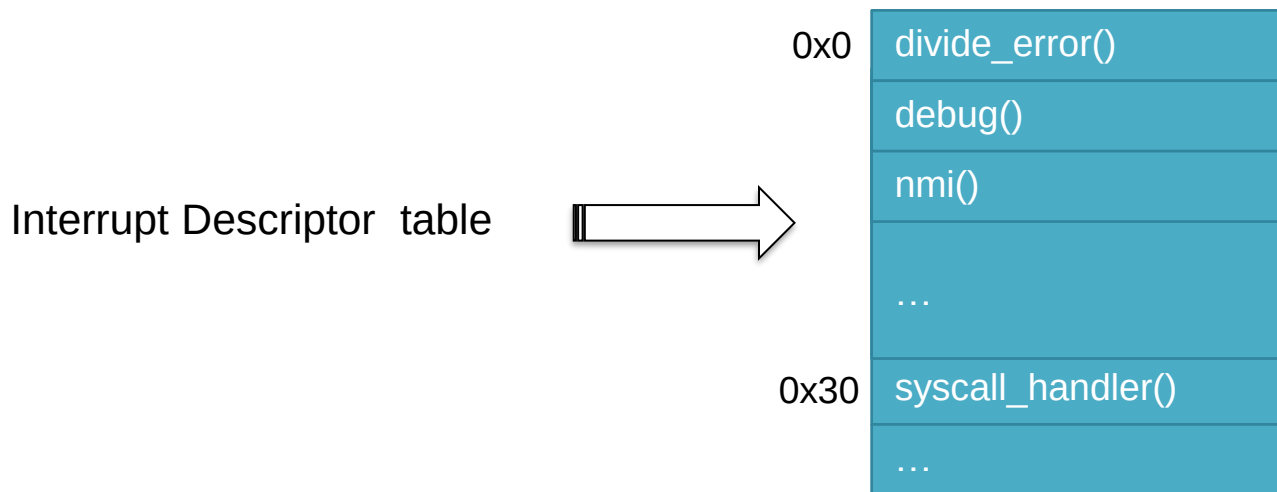
    /* Project 3 and optionally project 4. */
    SYS_MMAP,           /* Map a file into memory. */
    SYS_MUNMAP,         /* Remove a memory mapping. */

    /* Project 4 only. */
    SYS_CHDIR,          /* Change the current directory. */
    SYS_MKDIR,          /* Create a directory. */
    SYS_READDIR,        /* Reads a directory entry. */
    SYS_ISDIR,          /* Tests if a fd represents a directory. */
    SYS_INUMBER         /* Returns the inode number for a fd. */
};
```

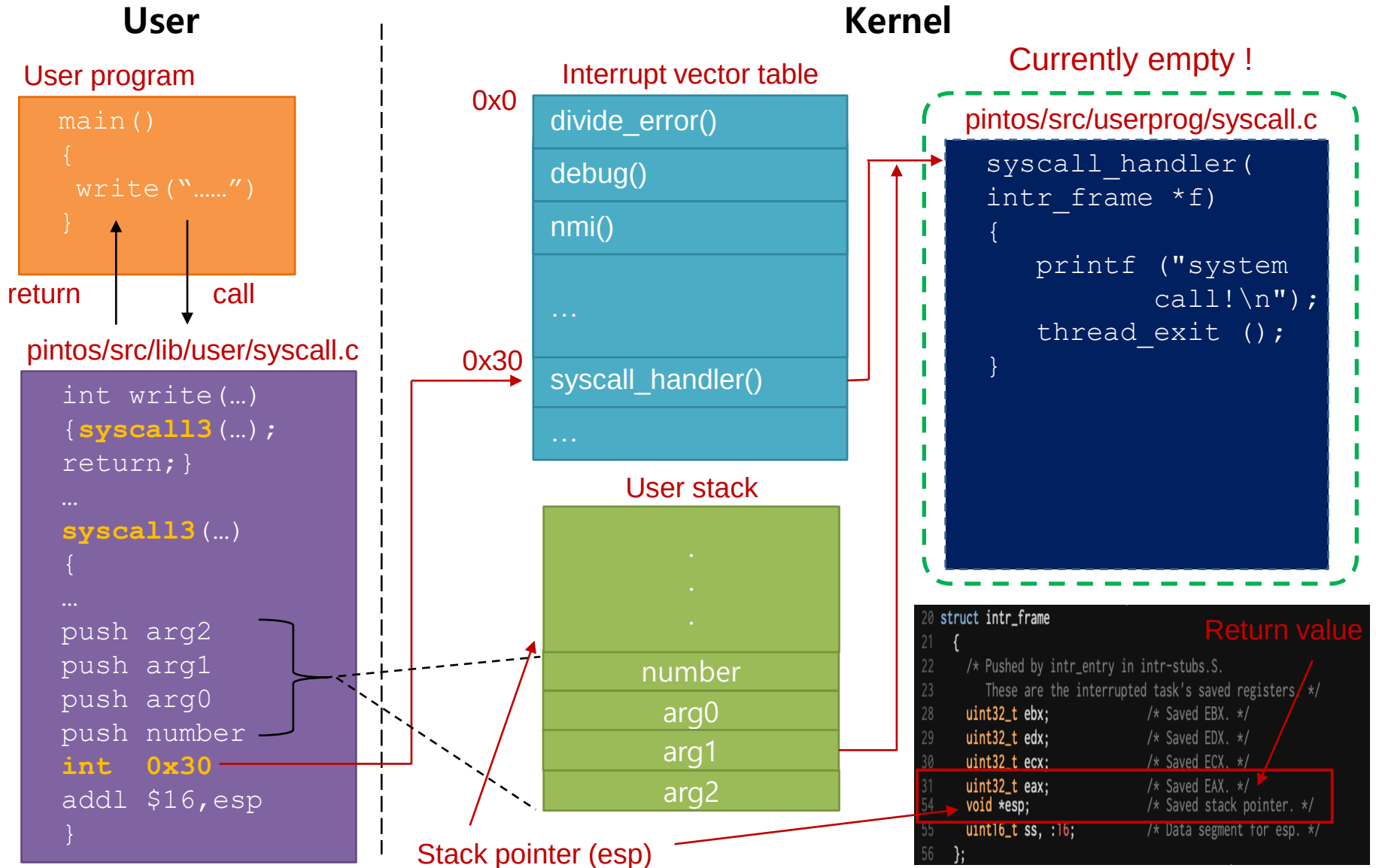
Example: Register System Call Handler at IDT

- **Interrupt Descriptor Table (IDT)** (or Interrupt Vector) is a table of the locations of the interrupt handlers for the associated interrupt number.
 - ◆ Register the system call handler at interrupt `0x30` (`src/userprog/syscall.c`).
 - ◆ This is done while Pintos was booting.

```
void  
syscall_init (void)  
{  
    intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");  
}
```



Example: `write()` Flow in Pintos (Kernel Side)



Example: Implementation of `write()` (`src/filesys.c`)

How do you bind the `write()` system call to `file_write()`?

```
off_t file_write (struct file *file, const void *buffer, off_t size) {  
    off_t bytes_written = inode_write_at (file->inode, buffer, size, file->pos);  
    file->pos += bytes_written;  
    return bytes_written;  
}
```

```
main()  
{  
    write(".....")  
}
```

```
int write(...)  
{ return  
  syscall3(...); }
```

```
divide_error()  
debug()  
nmi()  
...  
syscall_handler()  
...
```

```
syscall_handler(  
  intr_frame *f)  
{  
    ???  
}
```

```
file_write()
```

What Code to Run in Trap?

- ▣ Sources of trap (or interrupt)
 - ◆ Completion of disk IO
 - ◆ Keyboard interrupt
 - ◆ System call
- ▣ Trap handler: the code to run for each interrupt number (trap number).
- ▣ Trap table (IDT or Interrupt Vector Table)
 - ◆ Contains the addresses of the trap handlers.
 - ◆ The OS **informs the hardware of the locations** of trap handlers.
 - ◆ The instruction to inform the location of the trap table is also **privileged** instruction.
 - ◆ You cannot execute this instruction in user mode.

Solution for Problem 2: Cooperative Approach

- ❑ How can the OS regain control of the CPU so that it can switch between processes?
 - ◆ **Cooperative (or non-preemptive) approach:** Wait for system calls.
- ❑ Processes **periodically give up the CPU** by making system calls such as `yield`.
 - ◆ The OS decides to run some other task.
 - ◆ Applications also transfer control to the OS when they do something illegal.
 - Divide by zero
 - Try to access memory that it shouldn't be able to access
 - ◆ E.g., Early versions of the Macintosh OS, The old Xerox Alto system

**A process gets stuck in an infinite loop.
→ Reboot the machine.**

Solution for Problem 2: Non-Cooperative Approach

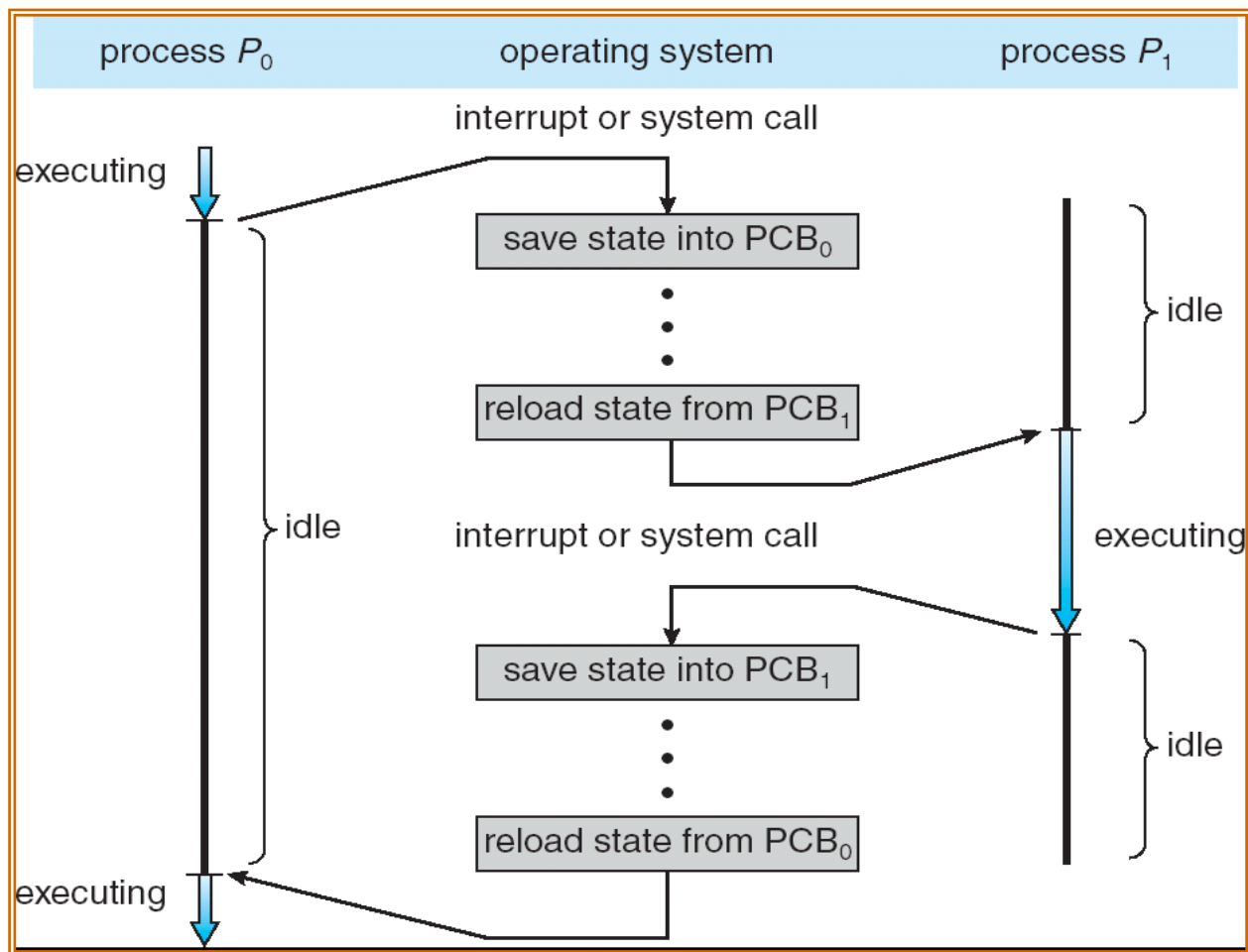
- ▣ How can the OS regain control of the CPU so that it can switch between processes?
 - ◆ **Non-cooperative (or preemptive) approach:** The OS takes control
- ▣ **Use a timer interrupt.**
 - ◆ During the boot sequence, the OS start the timer.
 - ◆ The timer raise an interrupt every few milliseconds.
 - ◆ When the interrupt is raised :
 - A pre-configured interrupt handler in the OS runs.
 - The currently running process is suspended.
 - Save enough of the state of the running process.
 - Restore the state of the new process.

A timer interrupt gives OS the ability to run again on a CPU.

Context Switch: Saving and Restoring Context

- The state of a process such as PC, the value of the CPU registers, the process state, and memory-management information, etc. is called a **context** of a process.
- When CPU switches to another process, the system must **save** the state of the old process onto its kernel stack (or PCB) and **restore** the saved state of the new process from the kernel stack (or PCB) scheduled to run.
 - ◆ For this, the OS executes the **context switch** code written generally by a low-level piece of assembly code.
- Context-switch time is pure overhead (i.e., system does no useful work while switching) and its speed varies from machine to machine.
 - ◆ A typical speed is a several microseconds.
 - ◆ Time depends on hardware support.
 - Some processors provide multiple sets of registers, which only requires changing the pointer to the current register sets.
 - Existence of special instructions such as a single instruction to load or store all registers.

Example: Context Switch



Example: The xv6 Context Switch Code

```
1 # void swtch(struct context **old, struct context *new);
2 #
3 # Save current register context in old
4 # and then load register context from new.
5 .globl swtch
6 swtch:
7     # Save old registers
8     movl 4(%esp), %eax           # put old ptr into eax
9     popl 0(%eax)                # save the old IP
10    movl %esp, 4(%eax)           # and stack
11    movl %ebx, 8(%eax)           # and other registers
12    movl %ecx, 12(%eax)
13    movl %edx, 16(%eax)
14    movl %esi, 20(%eax)
15    movl %edi, 24(%eax)
16    movl %ebp, 28(%eax)
17
18    # Load new registers
19    movl 4(%esp), %eax           # put new ptr into eax
20    movl 28(%eax), %ebp          # restore other registers
21    movl 24(%eax), %edi
22    movl 20(%eax), %esi
23    movl 16(%eax), %edx
24    movl 12(%eax), %ecx
25    movl 8(%eax), %ebx
26    movl 4(%eax), %esp          # stack is switched here
27    pushl 0(%eax)               # return addr put in place
28    ret                         # finally return into new ctxt
```

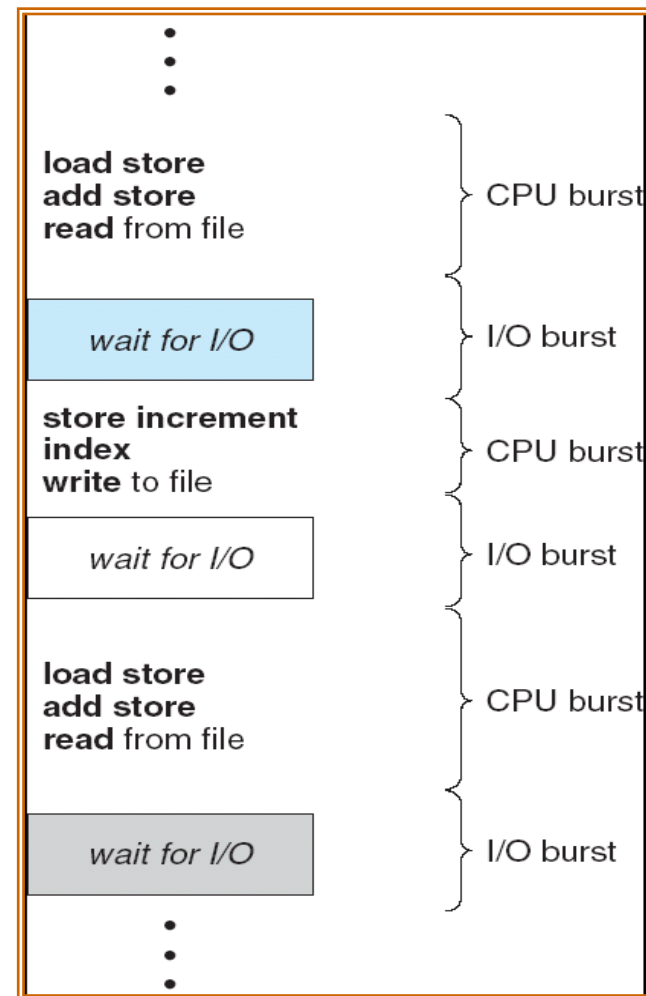
Worried About Concurrency?

- ❑ What happens if, during interrupt or trap handling, another interrupt occurs?
- ❑ OS handles these situations:
 - ◆ **Disable interrupts** during interrupt processing
 - ◆ Use a number of sophisticated **locking** schemes to protect concurrent access to internal data structures.
- ❑ This issue will be covered in the Concurrency section.

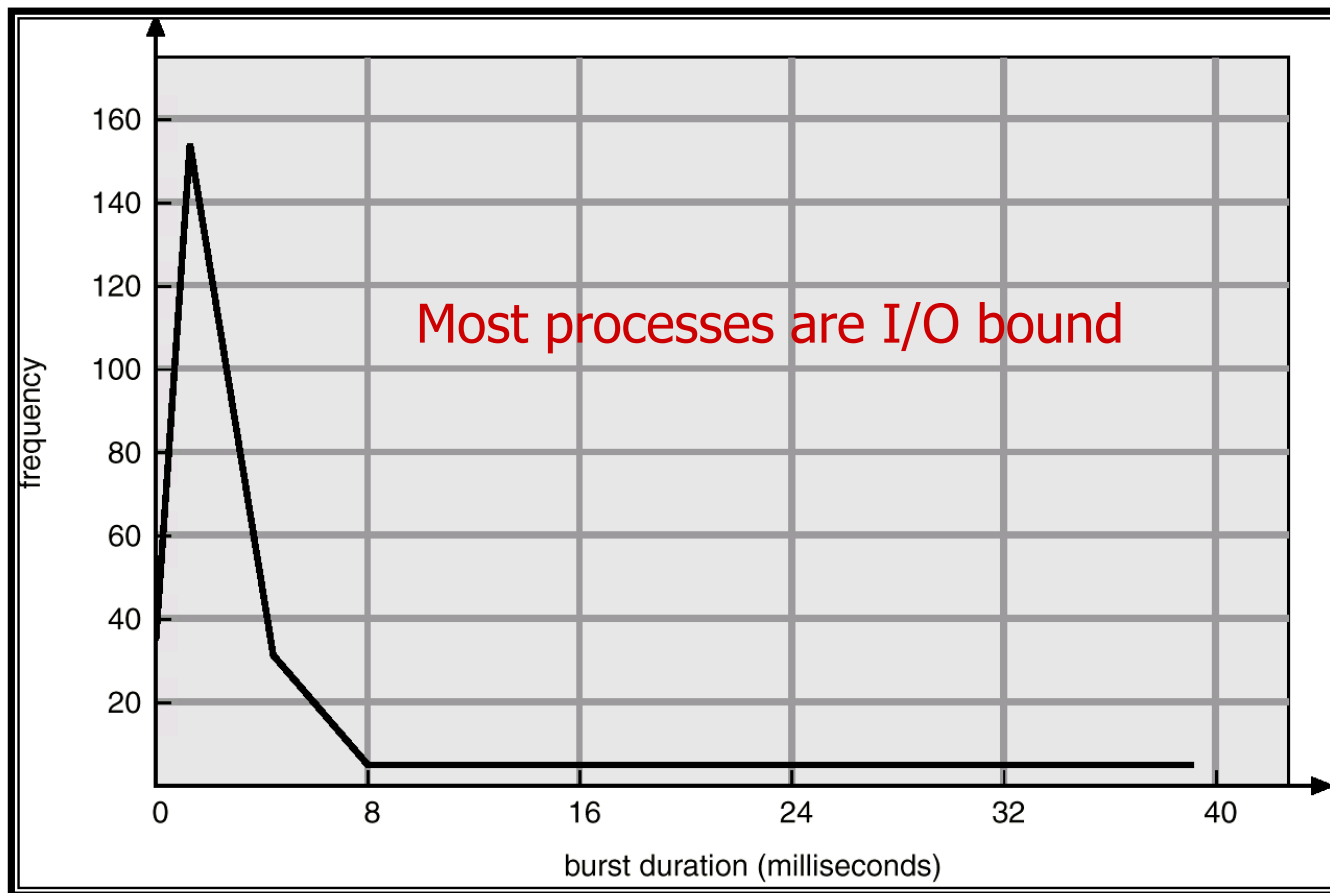
7. Scheduling: Introduction

Basic Concept

- ❑ To maximize CPU utilization, we need multiprogramming.
- ❑ But, with a single CPU core, only one process can run at a time.
- ❑ This requires CPU scheduling.
- ❑ Observed property of a process:
 - ◆ Process execution consists of a *cycle* of CPU execution and I/O wait (**CPU-I/O burst cycle**) and alternates this cycle.
 - ◆ CPU burst distribution is generally characterized as **exponential**, with many short CPU bursts, and a few long CPU bursts.



Histogram of CPU-burst Times



I/O bound process typically has many short CPU bursts, while a CPU bound process has a few long CPU bursts.

CPU Scheduler

- CPU scheduler selects a process to run among the processes in memory that are ready to execute, and allocates the CPU to one of them.
- CPU scheduling decisions may take place when a process:
 1. Switches from running to blocking state (i.e., when an I/O is initiated).
 2. Switches from running to ready state (i.e., when time quantum expires).
 3. Switches from blocking to ready (i.e., when an I/O completes).
 4. Terminates.
- When scheduling takes place **only** under 1 and 4, we say that the scheduling scheme is **non-preemptive** or **cooperative**.
- All other scheduling is **preemptive**. -> affect OS design !
 - ◆ Access to shared data, preemption while in kernel mode, etc.
 - ◆ Virtually all modern OSs use preemptive scheduling algorithms.

Scheduling Criteria

- ❑ **CPU utilization** – keep the CPU as busy as possible.
- ❑ **Throughput** – # of processes that complete their execution per time unit.
- ❑ **Turnaround time** – amount of time to execute a particular process (from submission to completion). This is the sum of the periods spent waiting in the ready queue, executing on the CPU, and doing I/O.
- ❑ **Waiting time** – amount of time a process has been waiting in the ready queue.
- ❑ **Response time** – amount of time it takes from when a request was submitted until the first response is produced, **not** output (for time-sharing environment).

Optimization Criteria

- ❑ In most cases, CPU scheduler needs to
 - ◆ **Maximize** CPU utilization and throughput.
 - ◆ **Minimize** turnaround time, waiting time and response time.
- ❑ In this case, the average values are optimized.
- ❑ However, under some circumstances, optimizing the minimum and maximum values is preferred.
- ❑ For interactive systems, it is sometimes more important to minimize the variance in the response time than to minimize the average response time.

Scheduling: Introduction

- Workload assumptions:

1. Each job runs for the **same amount of time**.
2. All jobs **arrive** at the same time.
3. All jobs only use the **CPU** (i.e., they perform no I/O).
4. The **run-time** of each job is known.

- Performance metric used in the class

- ♦ **Turnaround time** : The time at which **the job completes** minus the time at which **the job arrived** in the system.

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

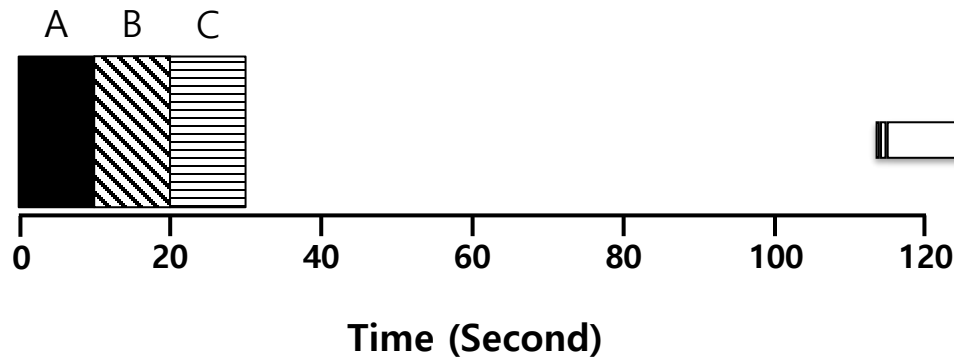
- ♦ **Waiting time** : The amount of time a process has been **waiting in the ready queue**.

- Another metric is **fairness**.

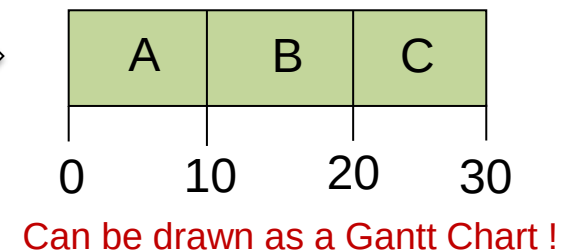
- ♦ Performance and fairness are often at odds in scheduling.

Algorithm 1: First In, First Out (FIFO)

- First Come, First Served (FCFS)
 - ◆ Non-preemptive scheduling algorithm.
 - ◆ Very simple and easy to implement.
- Example:
 - ◆ **A** arrived just before **B** which arrived just before **C**.
 - ◆ Each job runs for 10 seconds.



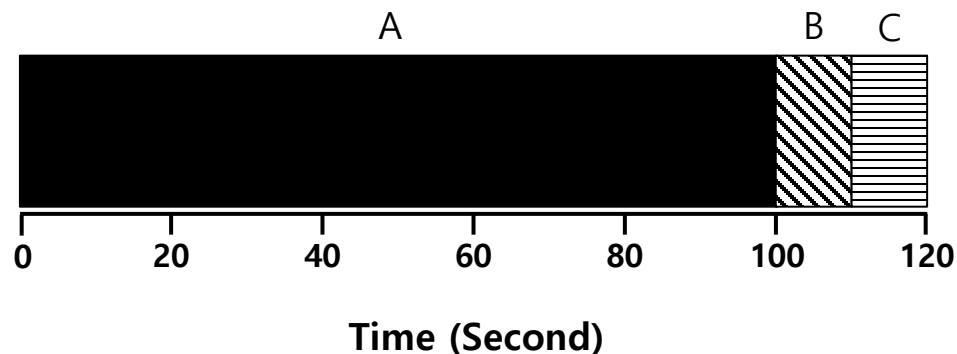
What about average waiting time ?
 $\frac{0 + 10 + 20}{3} = 10 \text{ sec}$



$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$

Why FIFO is Not That Great? – Convoy Effect

- Let's relax assumption 1: Each job no longer runs for the same amount of time.
- Example:
 - A arrived just before B which arrived just before C.
 - A runs for 100 seconds, B and C run for 10 each.



$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = 110 \text{ sec}$$

What is Convoy Effect and Why is Convoy Effect Bad? (1)

■ Convoy Effect

- ◆ Suppose there is one CPU intensive (large burst time) process in the ready queue, and several other processes with relatively less burst times but are Input/Output (I/O) bound (Need I/O operations frequently).
- ◆ The following then takes place –
 - The I/O bound processes are first allocated CPU time. As they are less CPU intensive, they quickly get executed and then go to I/O queues.
 - Now, the CPU intensive process is allocated CPU time. As its burst time is high, it takes time to complete.
 - While the CPU intensive process is being executed, the I/O bound processes complete their I/O operations and are moved back to ready queue.
 - However, the I/O bound processes are made to wait as the CPU intensive process still hasn't finished. **This leads to I/O devices being idle.**

What is Convoy Effect and Why is Convoy Effect Bad? (2)

❑ Convoy Effect (Continued)

- ◆ When the CPU intensive process gets over, it is sent to the I/O queue so that it can access and I/O device.
- ◆ Meanwhile, the I/O bound processes get their required CPU time and move back to I/O queue.
- ◆ However, they are made to wait because the CPU intensive process is still accessing an I/O device. As a result, **the CPU is sitting idle now.**

❑ Problem with FIFO Scheduler

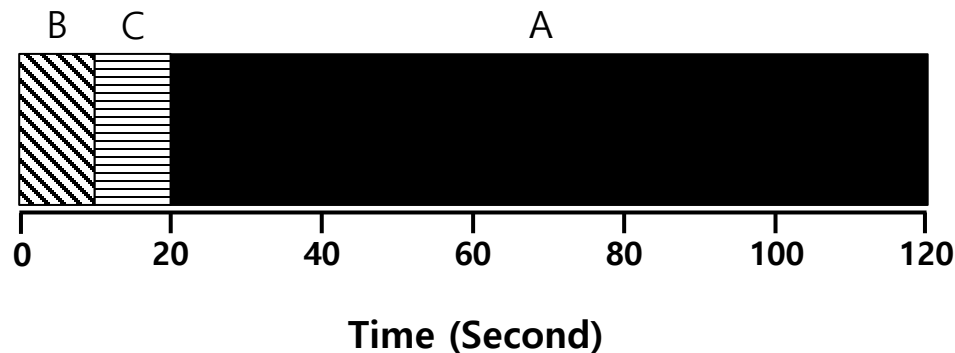
- ◆ FIFO : Turnaround time and waiting time can be extremely large when short jobs must wait for long jobs (because of convoy effect).

❑ Think about a new scheduler?

- ◆ SJF (Shortest Job First) : Choose job with smallest *run_time*.

Algorithm 2: Shortest Job First (SJF)

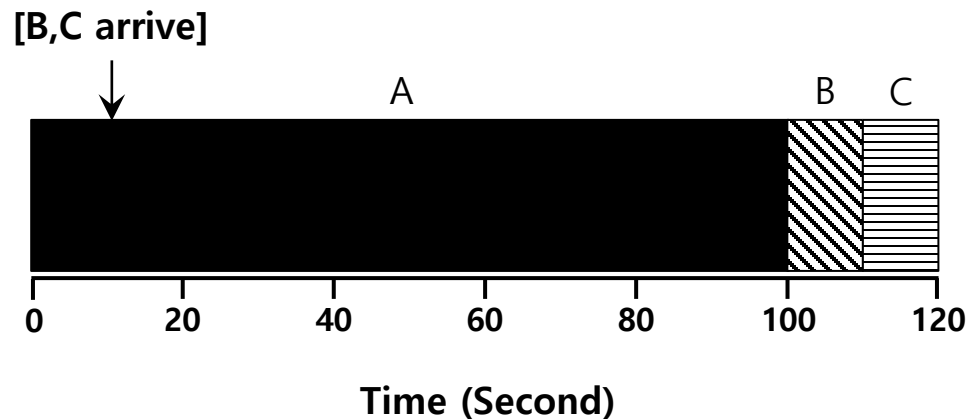
- Run the shortest job first, then the next shortest, and so on.
 - Non-preemptive scheduling algorithm.
- Example:
 - A** arrived just before **B** which arrived just before **C**.
 - A** runs for 100 seconds, **B** and **C** run for 10 each.



$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$

SJF with Late Arrivals from B and C

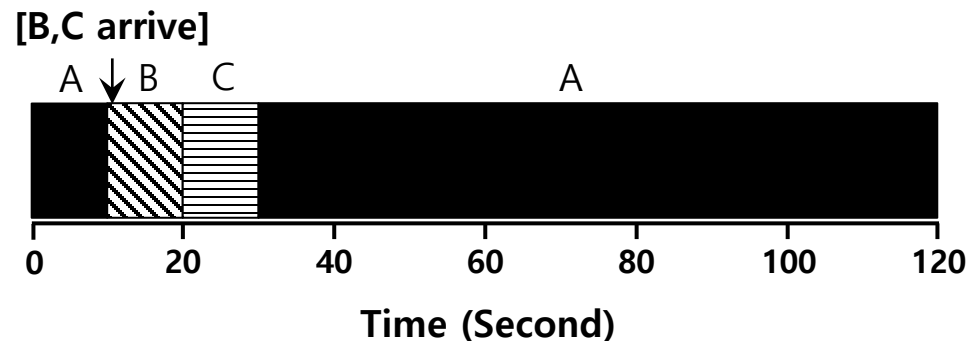
- ▣ Let's relax assumption 2: Jobs can arrive at any time.
- ▣ Example:
 - ◆ **A** arrives at $t = 0$ and needs to run for 100 seconds.
 - ◆ **B** and **C** arrive at $t = 10$ and each need to run for 10 seconds.



$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

Algorithm 3: Shortest Time-to-Completion First (STCF)

- Add **preemption** to SJF → **Shortest Time-to-Completion First (STCF)**.
 - ◆ Also known as **Preemptive Shortest Job First (PSJF)** or **Shortest Remaining Time First (SRTF)**.
- A new job enters the system:
 - ◆ Determine of the remaining jobs and new job.
 - ◆ Schedule the job which has the least time left.
- Example:
 - ◆ **A** arrives at $t = 0$ and needs to run for 100 seconds.
 - ◆ **B** and **C** arrive at $t = 10$ and each need to run for 10 seconds.



$$\text{Average turnaround time} = \frac{(120 - 0) + (20 - 10) + (30 - 10)}{3} = 50 \text{ sec}$$

Problem of SJF: Determining Next CPU Burst Length

- SJF is optimal in that it gives minimum average waiting and turnaround time for a given set of processes.
 - ◆ Moving a short process before a long one decreases the waiting time of the short processes more than it increases the waiting time of the long process.
 - ◆ The difficulty is knowing the length of the CPU burst → can **estimate** the length !
- The estimation can be done by using the length of previous CPU bursts, using **exponential averaging**.

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$
4. Define :

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

Examples of Exponential Averaging

- $\alpha = 0$

- ◆ $\tau_{n+1} = \tau_n$
- ◆ Recent history does not count.

- $\alpha = 1$

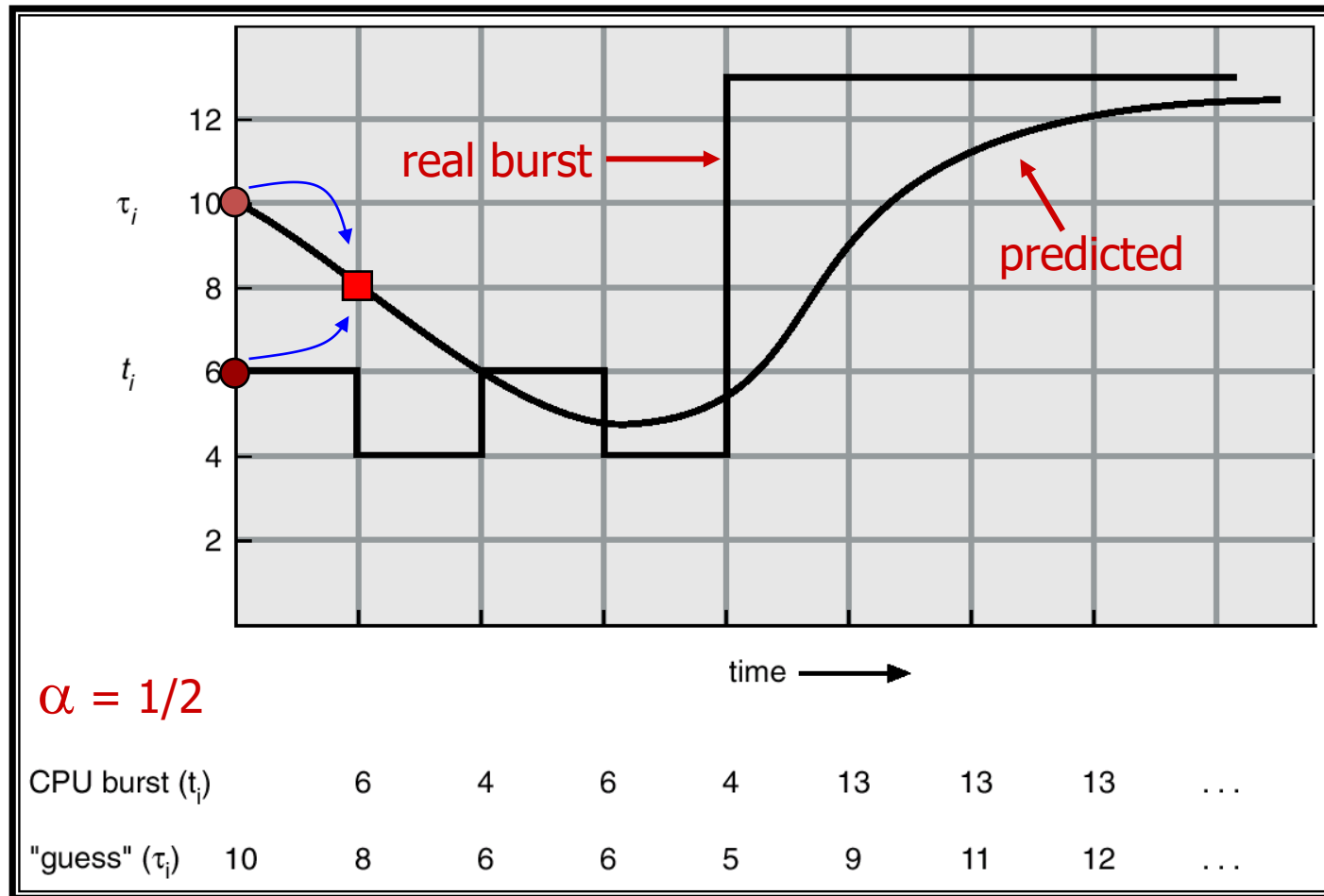
- ◆ $\tau_{n+1} = t_n$
- ◆ Only the actual last CPU burst counts.

- If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha) \alpha t_{n-1} \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor.

Example: Prediction of Next CPU Burst Length



New Scheduling Metric: Response Time

- ▣ The time from **when the job arrives** to the **first time it is scheduled**.

$$T_{response} = T_{firstrun} - T_{arrival}$$

- ◆ STCF and related disciplines are not particularly good for response time.

**How can we build a scheduler that is
sensitive to response time?**

Algorithm 4: Round Robin (RR) Scheduling

▣ Time-slicing Scheduling

- ◆ Each process gets a small unit of CPU time (**time slice** or **time quantum**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- ◆ The length of a time slice must be a *multiple* of the timer-interrupt period.

▣ If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1) \times q$ time units.

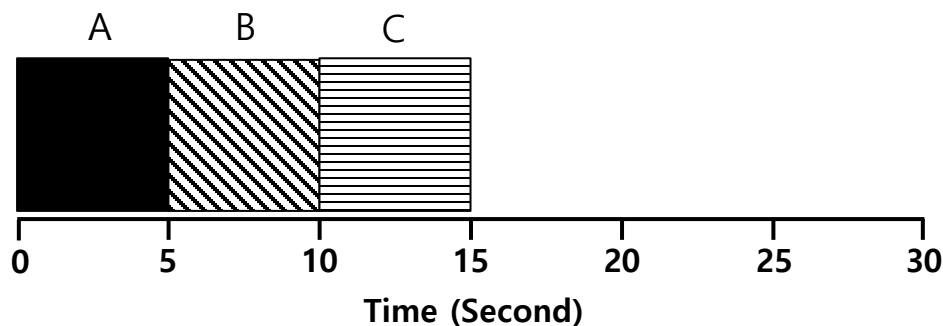
▣ Performance

- ◆ q large $\Rightarrow$ FCFS
- ◆ q small $\Rightarrow q$ must be large with respect to context switch.

RR is fair, but performs poorly on metrics such as turnaround time or waiting time !

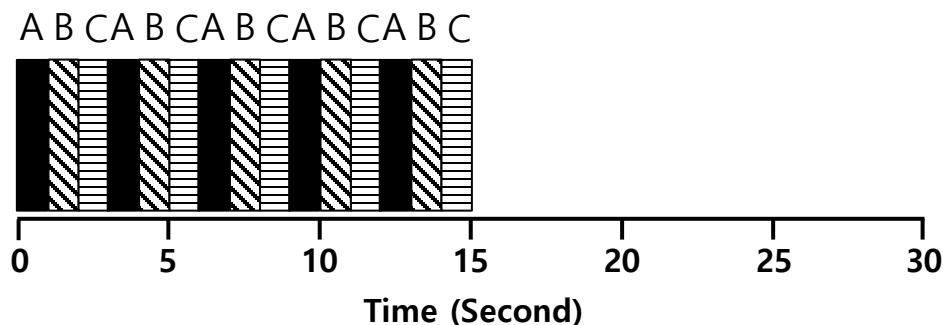
Example: RR Scheduling

- ▣ **A**, **B** and **C** arrive at the same time.
- ▣ They each wish to run for 5 seconds.



SJF (Bad for Response Time)

$$T_{average\ response} = \frac{0 + 5 + 10}{3} = \mathbf{5\ sec}$$



RR with a **time-slice of 1sec** (Good for Response Time)

$$T_{average\ response} = \frac{0 + 1 + 2}{3} = \mathbf{1\ sec}$$

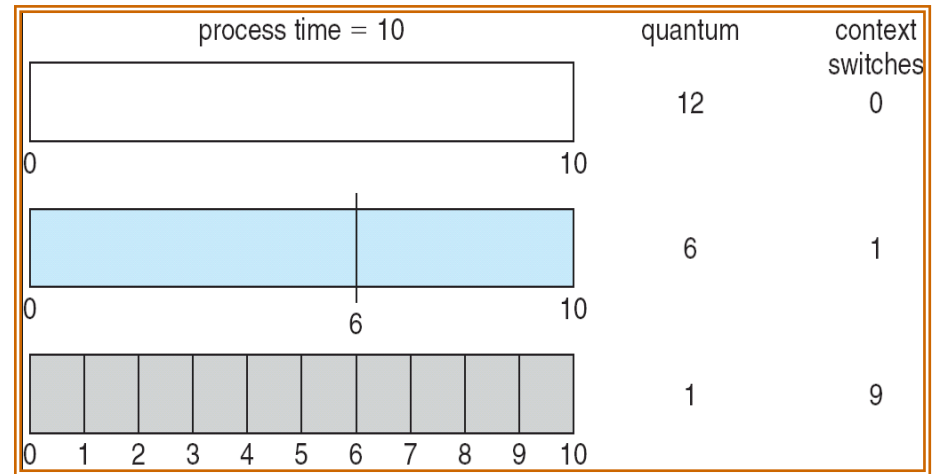
The Length of Time Slice is Critical

■ The shorter time slice

- ◆ Better response time.
- ◆ The cost of context switching will dominate overall performance.

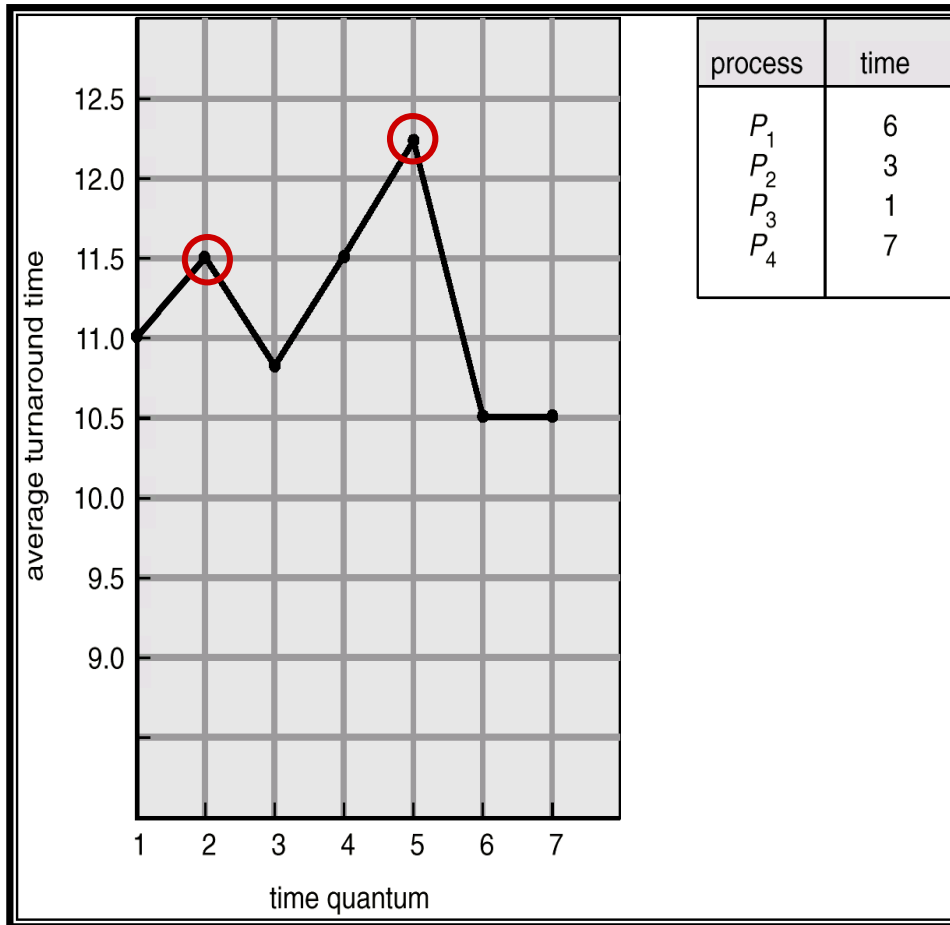
■ The longer time slice

- ◆ Amortize the cost of switching.
- ◆ Worse response time.



Deciding on the length of the time slice presents a **trade-off to a system designer !**

Time Slice vs Turnaround Time



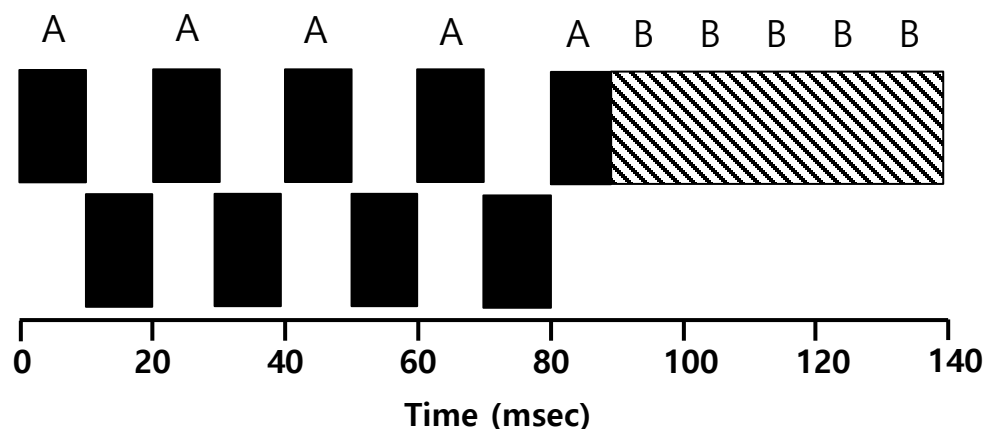
- If time slice is too large, RR scheduling becomes FCFS policy.
- Average turnaround time does not necessarily improve as the time-slice size increases.
- 80 percent of the CPU bursts should be shorter than the time slice.

Incorporating I/O (1)

- Let's relax assumption 3: All programs perform I/O.

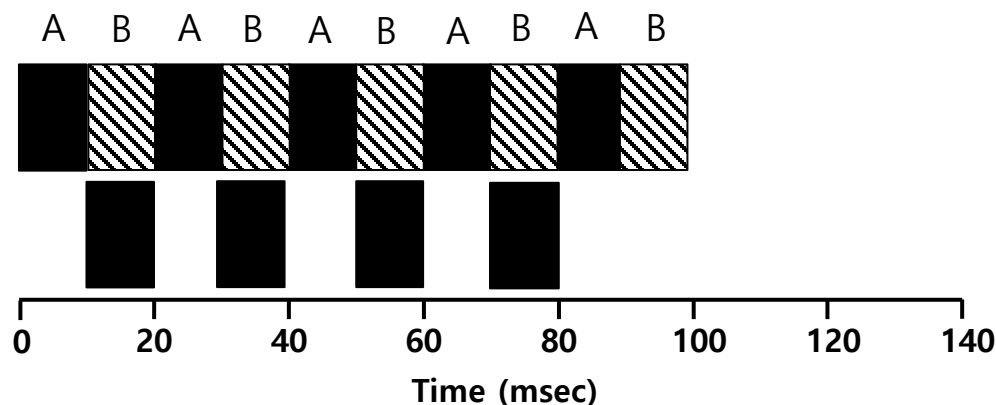
- Example:

- ◆ **A** and **B** need 50ms of CPU time each.
- ◆ **A** runs for 10ms and then issues an I/O request.
 - Each I/O takes 10ms.
- ◆ **B** simply uses the CPU for 50ms and performs no I/O.
- ◆ The scheduler runs **A** first, then **B** after.



Poor Use of Resources

Maximize CPU utilization



Overlap Allows Better Use of Resources

Incorporating I/O (2)

- ▣ When a job initiates an I/O request,
 - ◆ The job is blocked waiting for I/O completion.
 - ◆ The scheduler should schedule another job on the CPU.

- ▣ When the I/O completes,
 - ◆ An interrupt is raised.
 - ◆ The OS moves the process from blocked back to the ready state.

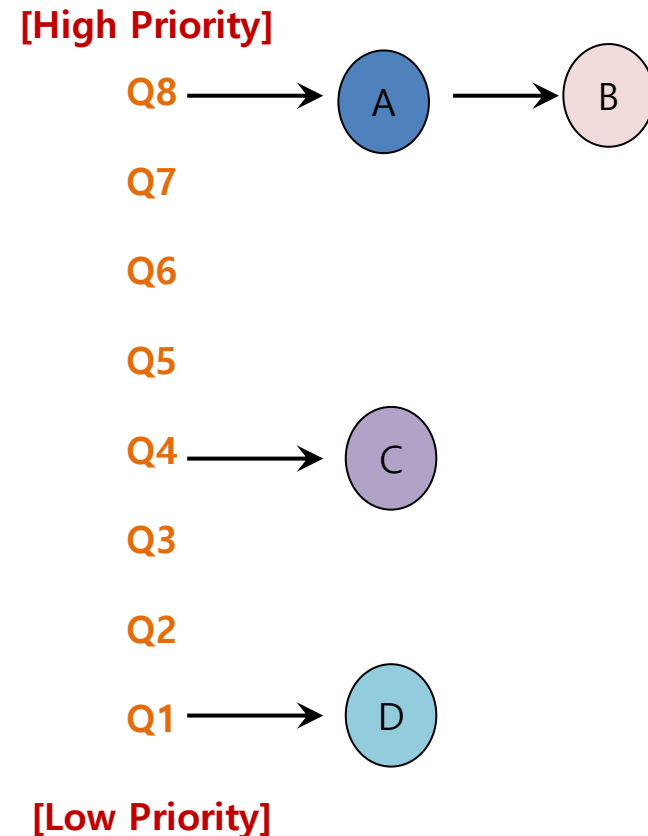
8: Scheduling: The Multi-Level Feedback Queue

Need a New and Efficient Algorithm

- ❑ FIFO algorithm
 - ◆ Simple but suffers from convoy effect → results in low CPU and device utilization.
- ❑ SJF scheduling algorithm
 - ◆ Good for reducing average turnaround time and average waiting time.
 - ◆ Problem: How to know the job length ? Is estimation always good?
- ❑ RR scheduling algorithm
 - ◆ Good for reducing response time but not good for reducing turnaround time.
- ❑ Need a new algorithm to:
 - ◆ **Optimize turnaround time** → Run shorter jobs first.
 - ◆ **Minimize response time** → Use RR to make processes run as much as possible.
 - ◆ **Learn from the past to predict the future** without a priori knowledge of job length.

Multi-Level Feedback Queue (MLFQ) Algorithm: Basic Rules

- MLFQ has a number of **distinct queues**.
 - ◆ Each queue has a **different priority level**.
- The execution order (priority) of a job
 - ◆ A job on a **higher queue** is chosen to run.
 - ◆ Use **round-robin** scheduling among jobs in the same queue.
 - ◆ **Rule 1**: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
 - ◆ **Rule 2**: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
- MLFQ varies the priority of a job based on **its observed behavior**.
 - ◆ A job repeatedly relinquishes the CPU while waiting IOs → Keep its priority high.
 - ◆ A job uses the CPU intensively for long periods of time → Reduce its priority.



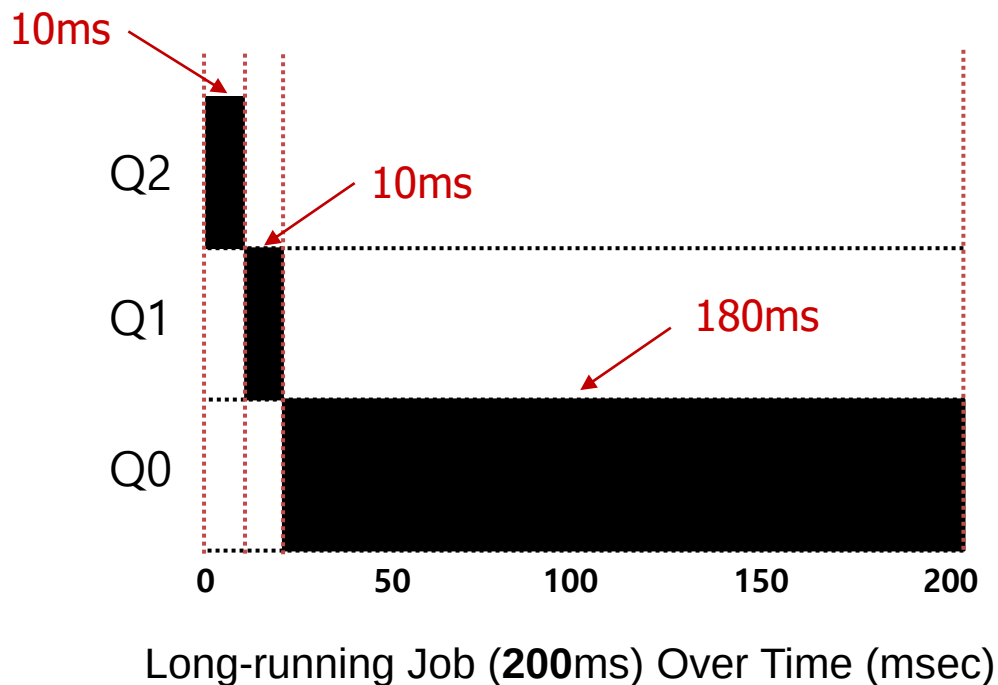
MLFQ: How to Change Priority

- ▣ MLFQ priority adjustment algorithm:
 - ◆ **Rule 3:** When a job enters the system, it is placed at the **queue with the highest priority**.
 - ◆ **Rule 4a:** If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down on queue).
 - ◆ **Rule 4b:** If a job gives up the CPU before the time slice is up, it stays at the same priority level.

In this manner, MLFQ approximates SJF !

Example 1: A Single Long-Running Job

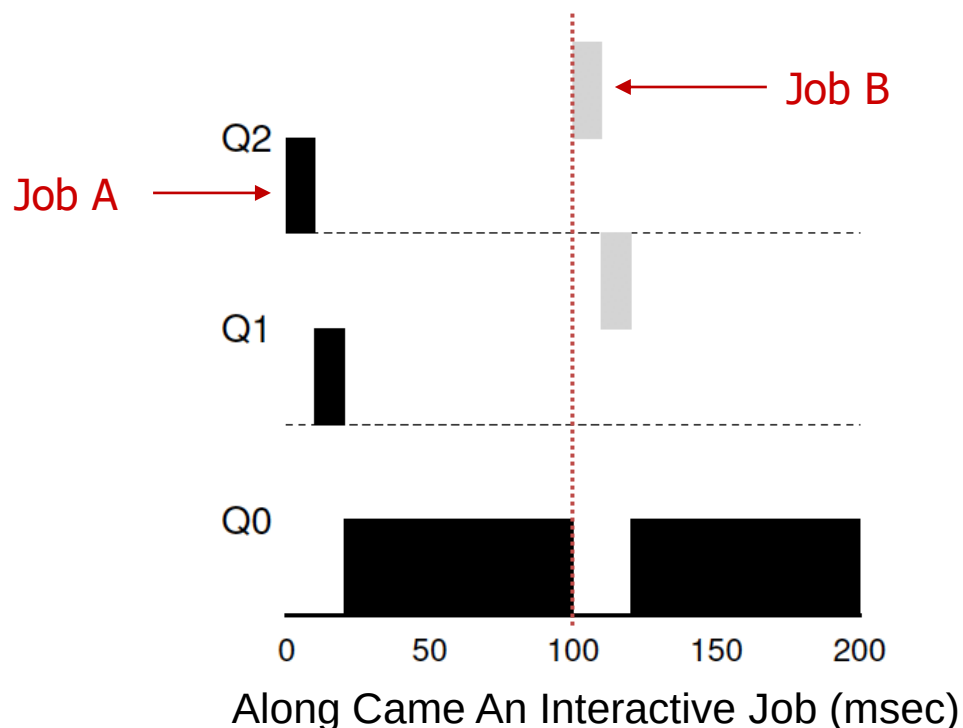
- ▣ A three-queue scheduler with time slice **10ms**.



Example 2: Along Came a Short Job

Assumption:

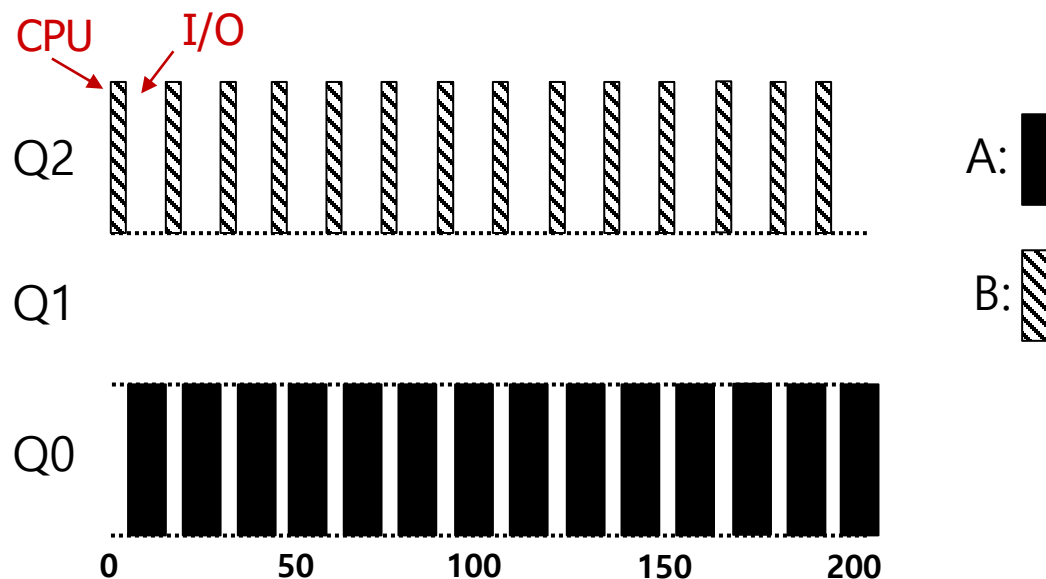
- ♦ **Job A:** A long-running CPU-intensive job.
- ♦ **Job B:** A short-running interactive job (**20ms** runtime).
- ♦ **A** has been running for some time, and then **B** arrives at time $T = 100$.



Example 3: What About I/O?

Assumption:

- ♦ **Job A:** A long-running CPU-intensive job.
- ♦ **Job B:** An interactive job that need the CPU only for 1ms before performing an I/O.



A Mixed I/O-intensive and CPU-intensive Workload (msec)

The MLFQ approach keeps an **interactive job** at the **highest priority**.

Problems with the Basic MLFQ

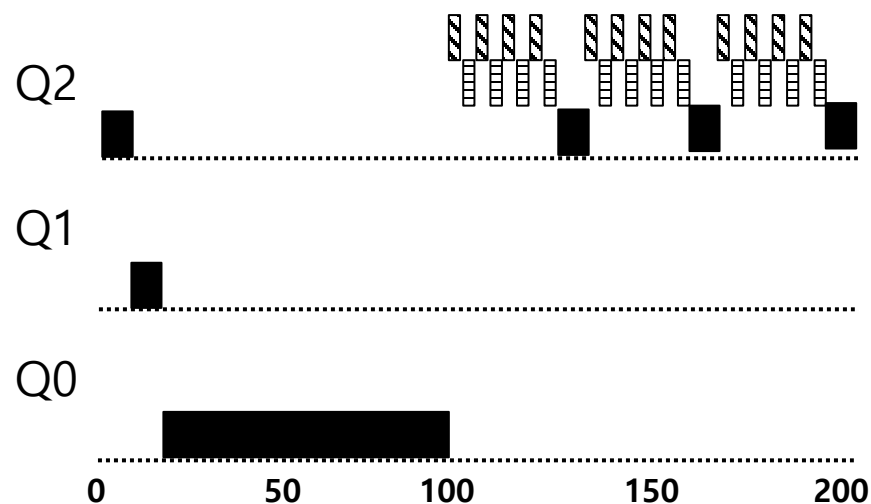
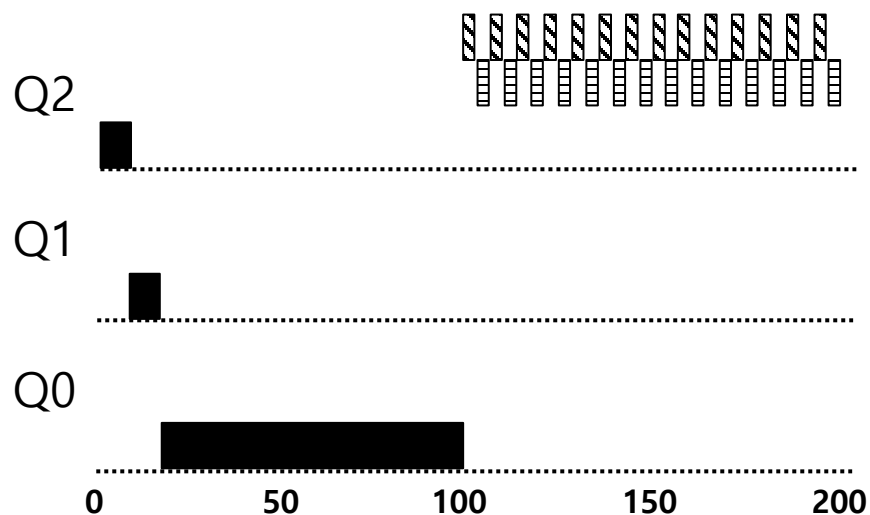
- ❑ Starvation
 - ◆ If there are “too many” interactive jobs in the system.
 - ◆ Long-running jobs will never receive any CPU time.
- ❑ Game the scheduler
 - ◆ After running 99% of a time slice, issue an I/O operation.
 - ◆ The job gain a higher percentage of CPU time.
- ❑ A program may change its behavior over time.
 - ◆ CPU bound process → I/O bound process.
 - ◆ Need priority boosting mechanism.

The Priority Boost

- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.

- ◆ Example:

- A long-running job(A) with two short-running interactive job(B, C)

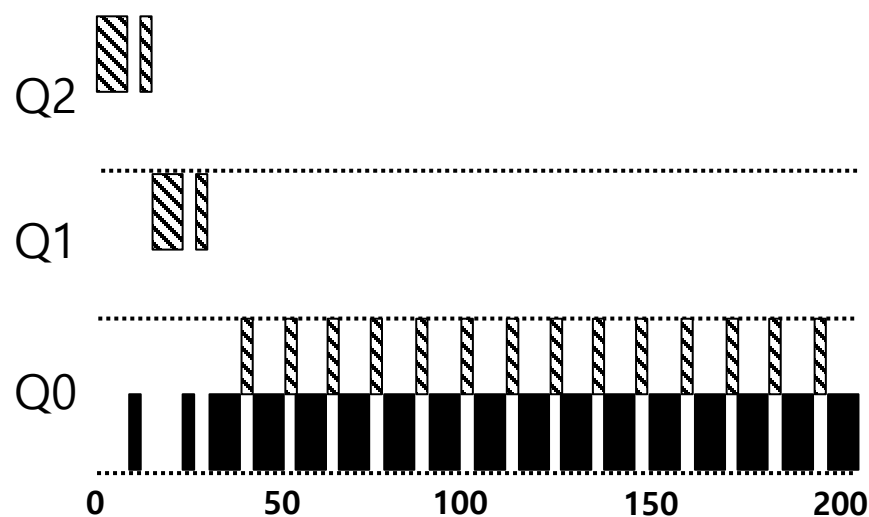
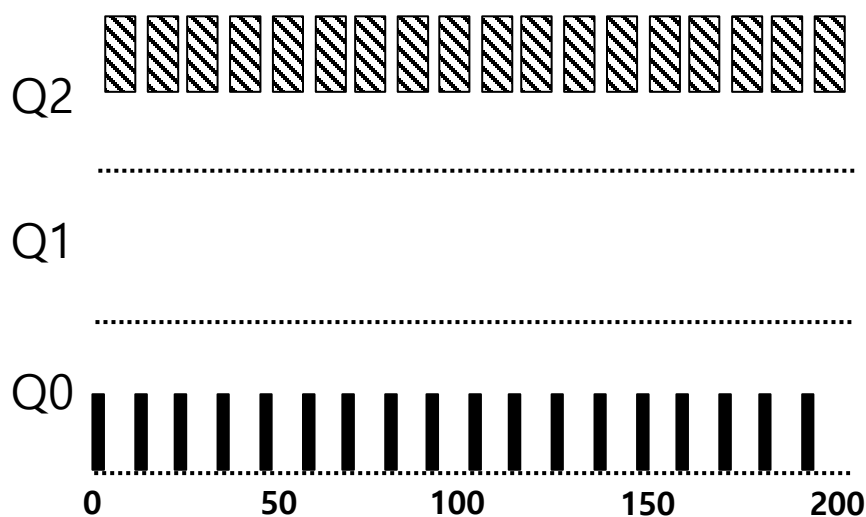


Without(Left) and With(Right) Priority Boost

A:  B:  C: 

Better Accounting

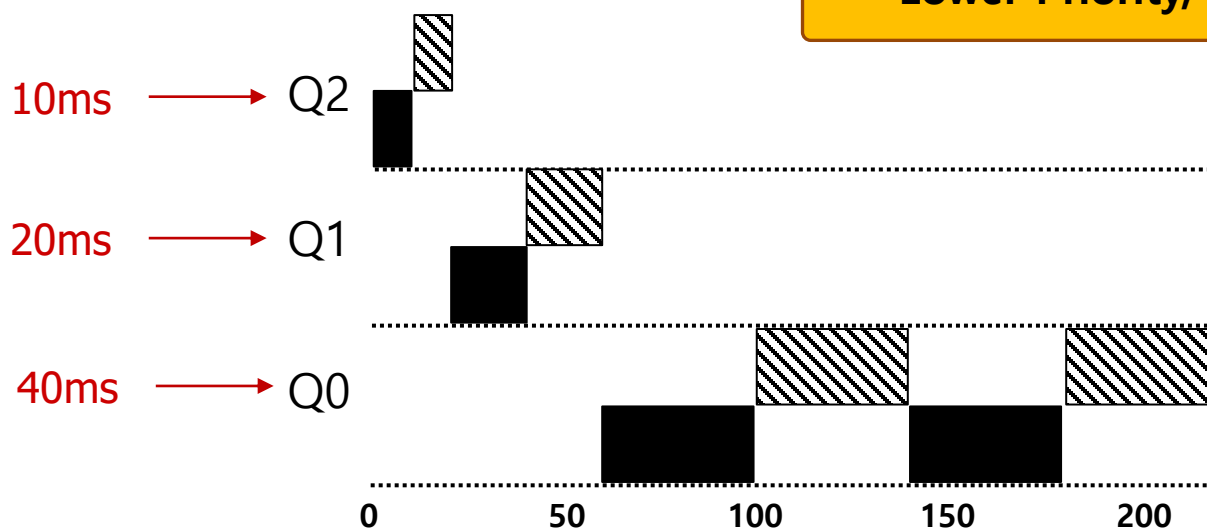
- ▣ How to prevent gaming of our scheduler?
- ▣ Solution:
 - ◆ **Rule 4** (Rewrite Rules 4a and 4b): Once a job **uses up its time allotment** at a given level (regardless of how many times it has given up the CPU), **its priority is reduced** (i.e., it moves down on queue).



Without(Left) and With(Right) Gaming Tolerance

Tuning MLFQ And Other Issues

- ◆ The high-priority queues → Short time slices.
 - E.g., 10 or fewer milliseconds
- ◆ The Low-priority queue → Longer time slices.
 - E.g., 100 milliseconds



Example) **10ms** for the **highest** queue, **20ms** for the **middle**, **40ms** for the **lowest**

MLFQ: Summary

▣ The refined set of MLFQ rules:

- ◆ **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- ◆ **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
- ◆ **Rule 3:** When a job enters the system, it is placed at the highest priority.
- ◆ **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down on queue).
- ◆ **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.

▣ Beauty of MLFQ

- ◆ **It does not require prior knowledge** on the CPU usage of a process.

The Solaris MLFQ implementation

- ▣ For the Time-Sharing scheduling class (TS)
 - ◆ 60 Queues.
 - ◆ Slowly increasing time-slice length.
 - The highest priority: 20msec.
 - The lowest priority: A few hundred milliseconds.
 - ◆ Priorities boosted around every 1 second or so.

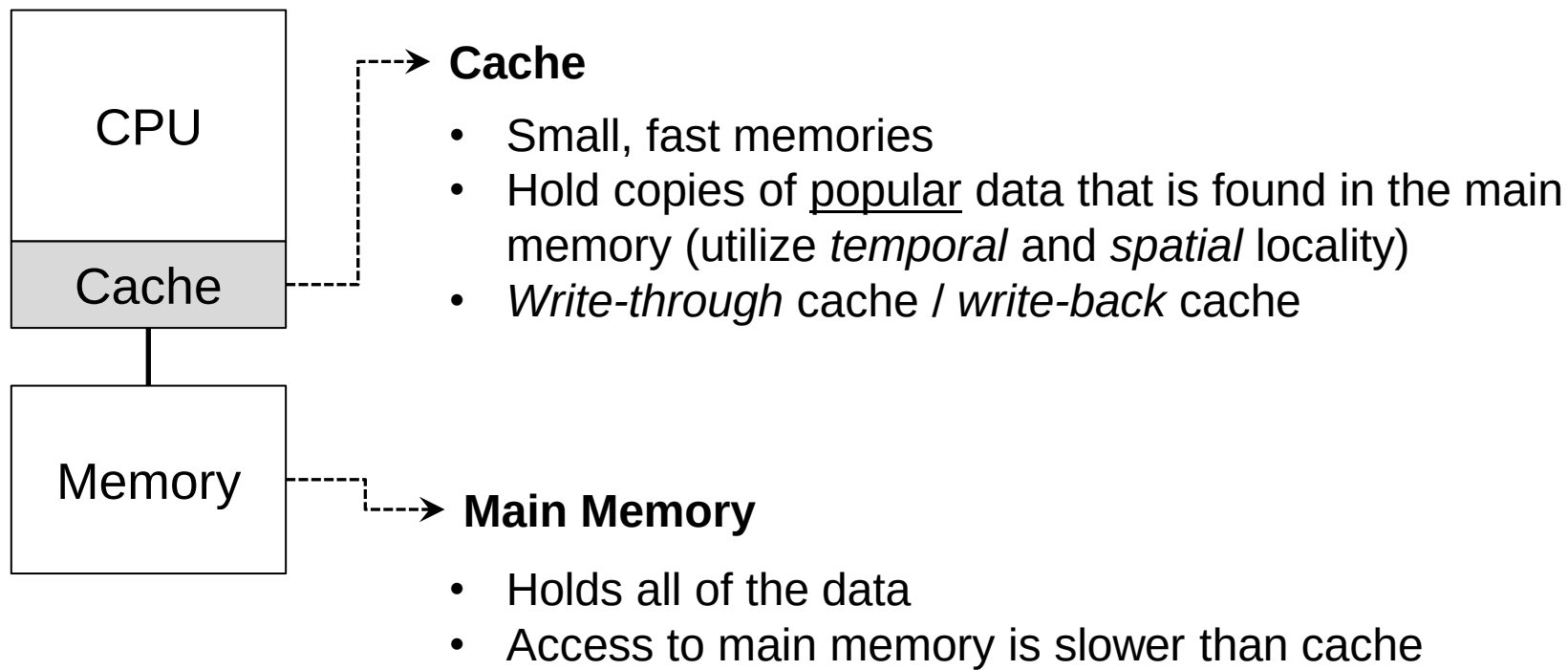
10. Multiprocessor Scheduling (Advanced)

Multiprocessor Scheduling

- ▣ The rise of the **multicore processor** is the source of multiprocessor-scheduling proliferation.
 - ◆ **Multicore**: Multiple CPU cores are packed onto a single chip.
- ▣ Adding more CPUs does not make that single application run faster. → You'll have to rewrite application to run in parallel, using **threads**.

How to schedule jobs on **multiple CPUs (or cores)**?

Background: Single CPU with Cache

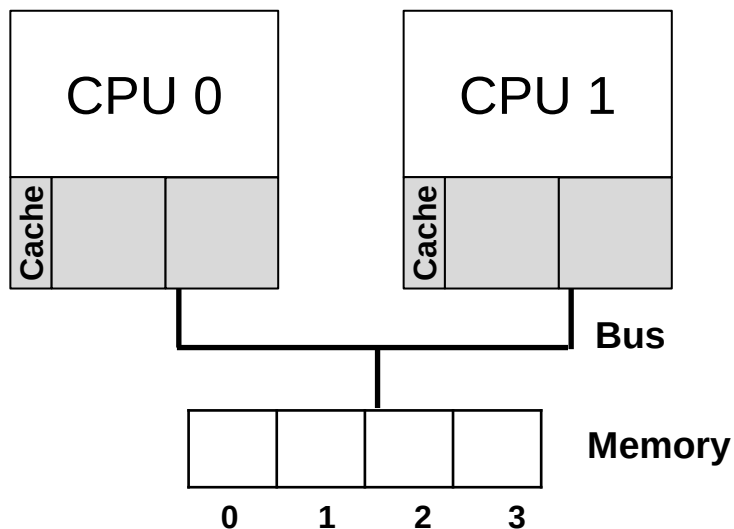


By keeping data in cache, the system can make slow memory
appear to be a fast one

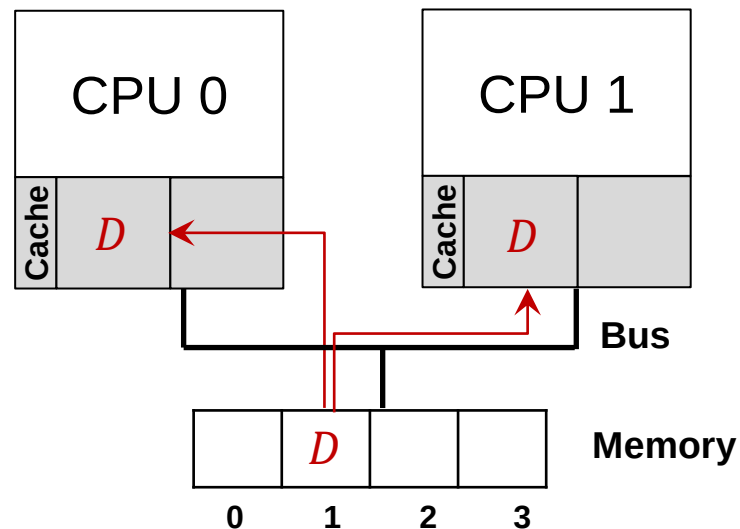
Background: Cache Coherence (1)

- Consistency of shared resource data stored in multiple caches.

0. Two CPUs with caches sharing memory.

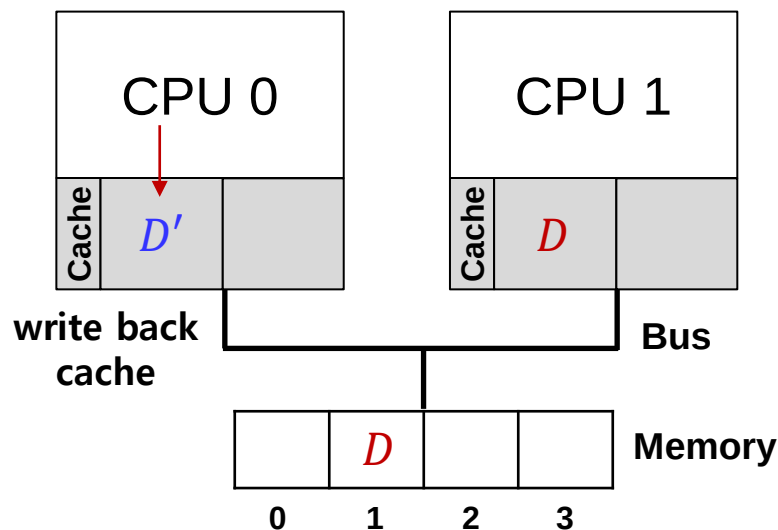


1. CPU0/CPU1 both **read** *D* at address 1.

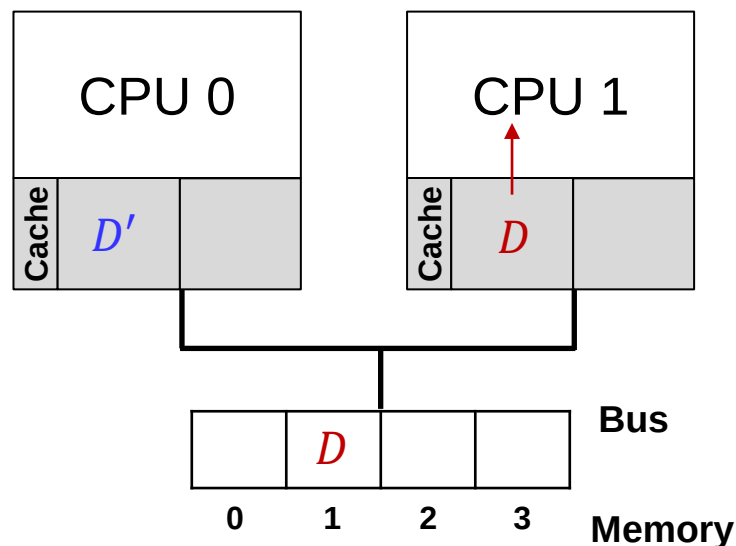


Background: Cache Coherence (2)

2. CPU0 **updates** D and CPU1 is scheduled.



3. CPU1 **re-reads** D in the cache.



CPU1 gets the old value D instead of the correct value D' .

❑ Bus snooping protocol

- ◆ Cache controller **broadcasts** its activity through bus.
- ◆ Each cache pays attention to memory updates by **snooping the bus**.
- ◆ When a CPU sees an update for a data item it holds in its cache, it will notice the change and either *invalidate* its copy or *update* it.

Background: Don't Forget Synchronization

- When accessing shared data across CPUs, **mutual exclusion** primitives should likely be used to guarantee correctness.

Simple List Delete Code

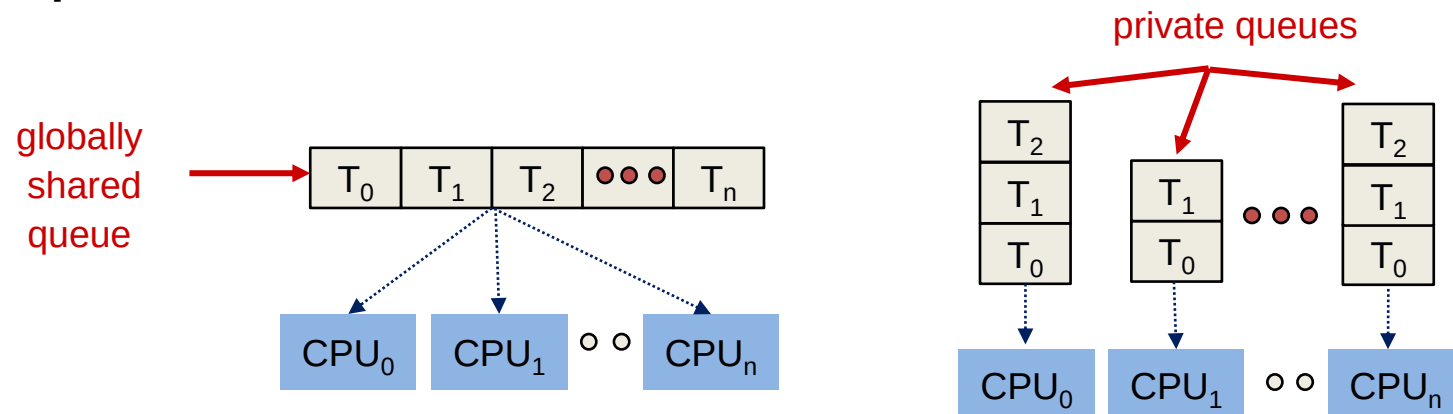
```
1      pthread_mutex_t m;  
2      typedef struct __Node_t {  
3          int value;  
4          struct __Node_t *next;  
5      } Node_t;  
6  
7      int List_Pop() {  
8          lock(&m);  
9          Node_t *tmp = head;           // remember old head ...  
10         int value = head->value;       // ... and its value  
11         head = head->next;             // advance head to next pointer  
12         free(tmp);                     // free old head  
13         unlock(&m);  
14         return value;                  // return value at head  
15     }
```

global variable



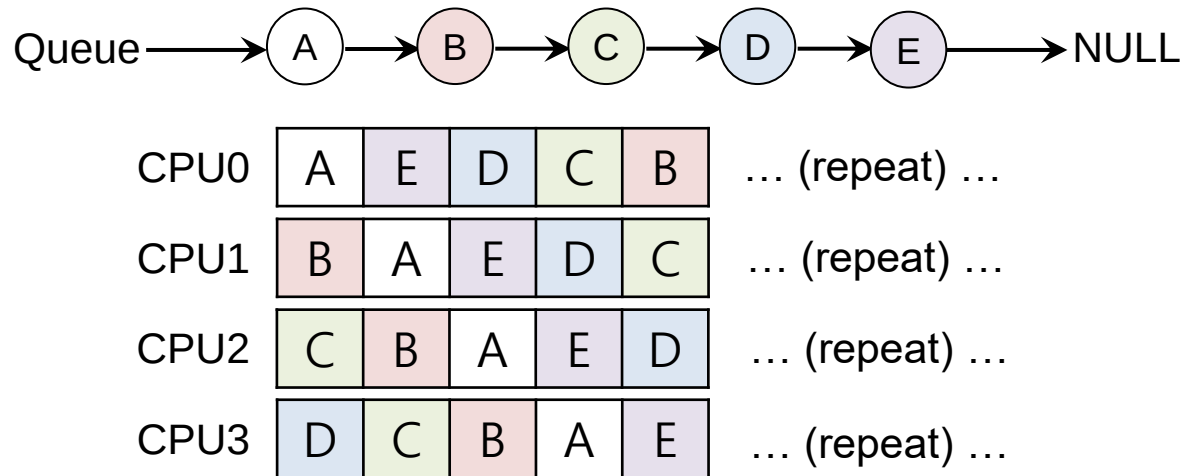
Asymmetric vs Symmetric Multiprocessing Scheduling

- ❑ CPU scheduling is more complex when multiple CPUs are available.
- ❑ Assume that CPUs are identical – **homogeneous** – in terms of their functionality for simplicity.
- ❑ Two approaches are possible :
 - ◆ **Asymmetric multiprocessing** – only one CPU accesses the system data structures, alleviating the need for data sharing.
 - ◆ **Symmetric multiprocessing (SMP)** – each CPU is self-scheduling by accessing and updating a **globally shared queue** or **multiple private queues**.



Single Queue Multiprocessor Scheduling (SQMS)

- Put all jobs that need to be scheduled into **a single queue**.
 - Each CPU simply picks the next job from the **globally shared queue**.
 - Example:

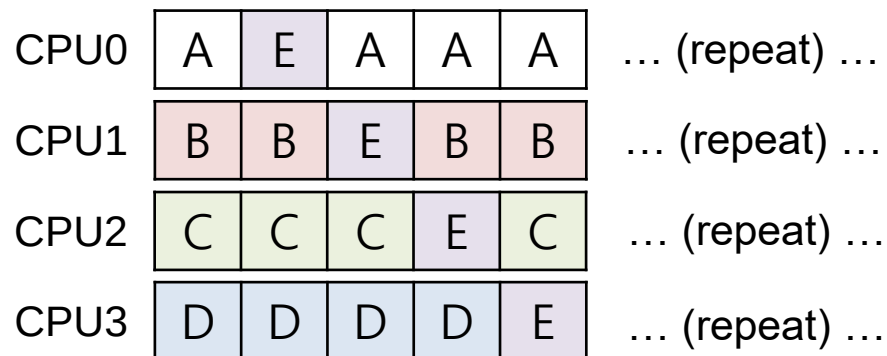
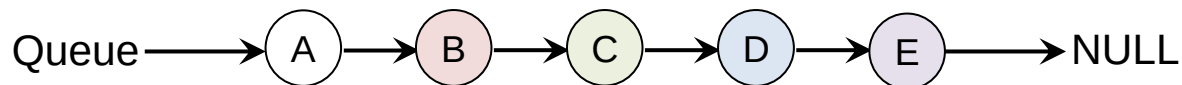


- SQMS needs some form of locking → Lack of scalability**
- Jobs move too frequently across different queues !**

Cache Affinity (Processor Affinity)

- Consider what happens to cache memory when a process has been running on a specific CPU.
 - ◆ A process builds up a fair bit of state in the cache of a CPU.
 - ◆ The next time the process run, it will run faster if some of its state is already present in the cache on that CPU.
 - ◆ If a process is migrated, the contents of cache memory in the source CPU is useless and the destination CPU needs to populate the contents again.
- Needs **cache (processor) affinity** that keeps a process running on the same CPU.
- Two possibilities :
 - ◆ **Soft affinity** : Attempt to keep a process running on the same CPU but not guarantee it – possible to migrate.
 - ◆ **Hard affinity** : Prohibit migration – provide **system calls** to specify that the process should not be migrated (e.g., Linux).
- A multiprocessor scheduler should consider cache affinity when making its scheduling decision.

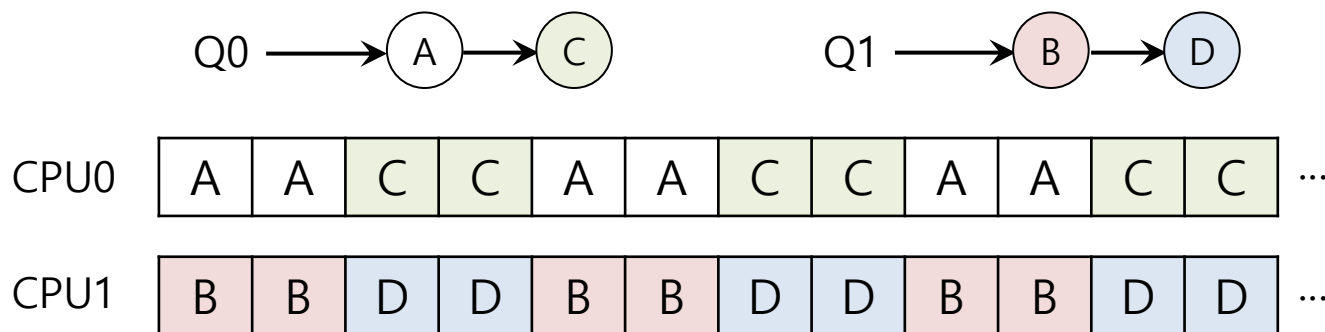
Example with Cache Affinity



- ◆ Preserve affinity as much as possible.
 - Jobs A through D are not moved across CPUs.
 - Only job E is migrating from CPU to CPU.
- ◆ Implementing such a scheme can be **complex**.

Multi-Queue Multiprocessor Scheduling (MQMS)

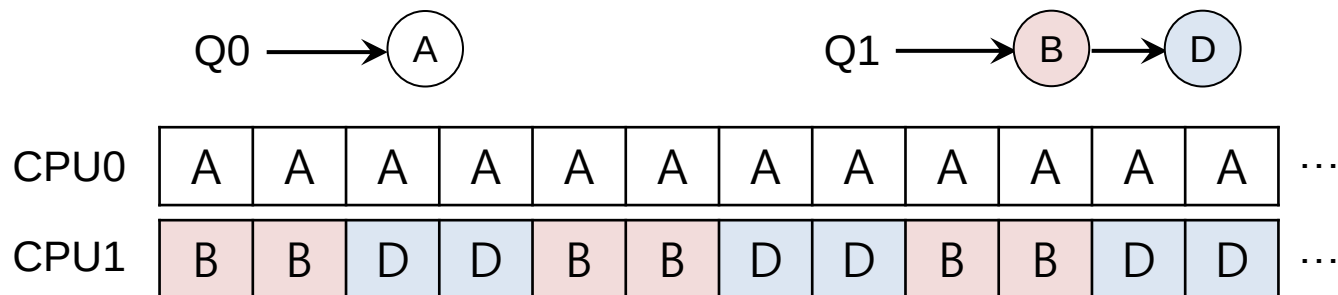
- MQMS consists of **multiple scheduling queues**.
 - Each queue will follow **a particular scheduling discipline**.
 - When a job enters the system, it is placed on **exactly one** scheduling queue.
 - Avoid the problems of information sharing and synchronization.
- With **round robin**, the system might produce a schedule that looks like this:



MQMS provides more scalability and cache affinity !

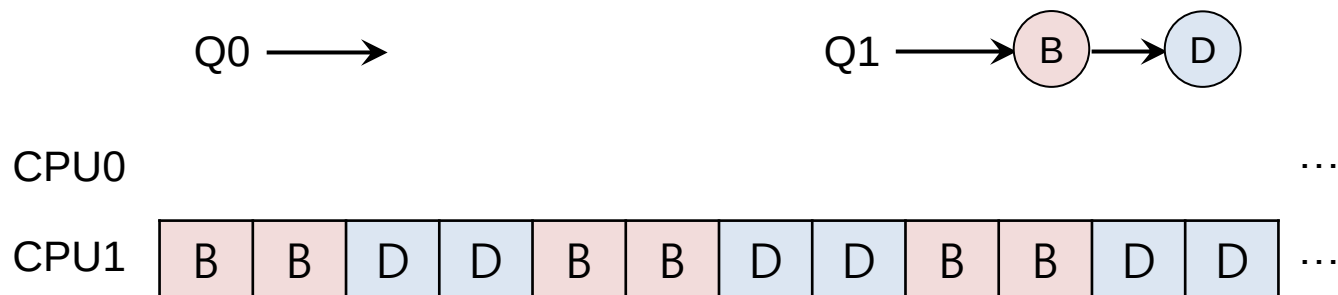
Load Imbalance Issue of MQMS

- After job C in Q0 finishes:



A gets twice as much CPU as B and D !

- After job A in Q0 finishes:



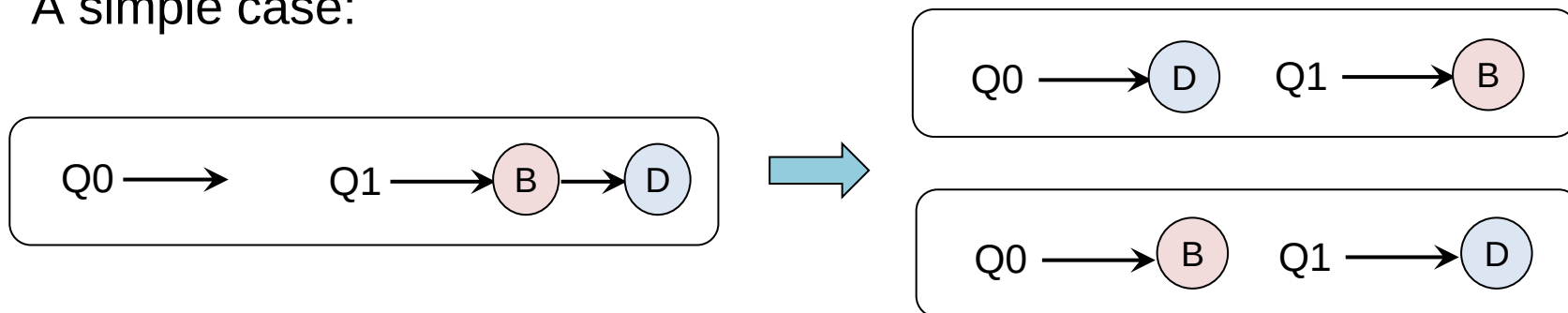
CPU0 will be left idle !

Load Balancing Algorithm in MQMS

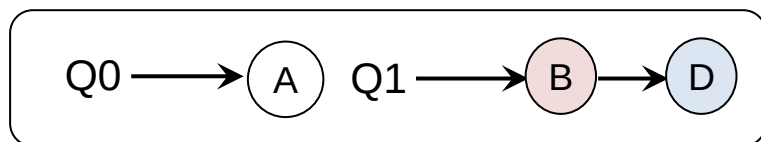
- **Load balancing** attempts to keep the workload evenly distributed across all CPUs.
 - ◆ Load balancing is typically only necessary on systems where each processor has its own private queue (i.e., multiple scheduling queues).
- Most contemporary OSs supporting SMP have a private queue of eligible CPUs.
- Two general approaches :
 - ◆ **Push migration** : A specific task periodically checks the load on each CPU and distribute the load evenly .
 - ◆ **Pull migration** : An idle CPU pulls a waiting task from a busy CPU.
- Both approaches can be implemented at the same time (i.e., Linux runs its load balancing algorithm every 200ms (push migration) or whenever the run queue for a processor is empty (pull migration)).

Example with Load Balancing

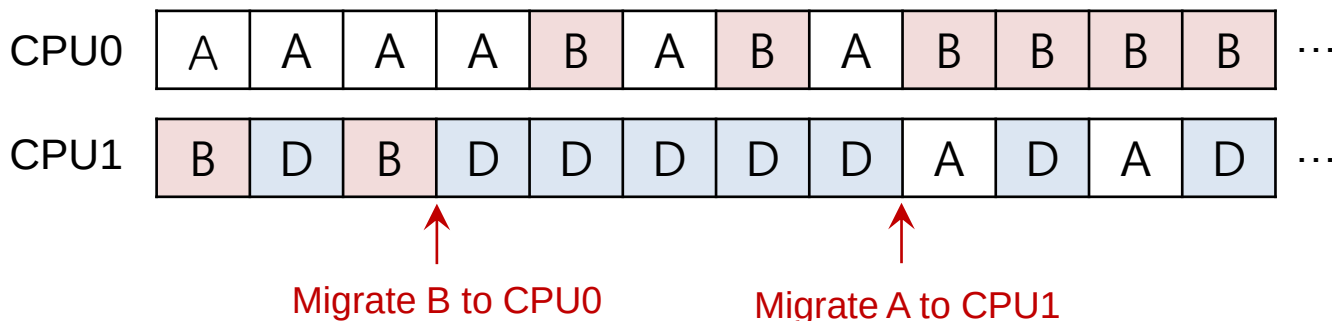
- A simple case:



- A more tricky case:



Keep switching jobs !



Work Stealing

- ▣ Move jobs between queues

- ◆ Implementation:

- A source queue that is low on jobs is picked.
 - The source queue occasionally peeks at another target queue.
 - If the target queue is more full than the source queue, the source will “**steal**” one or more jobs from the target queue.

- ◆ Cons:

- High overhead and trouble in scaling.